Björn Thor, Kaden, Sondre

Introduction {-}

Welcome to MS&E 244: Statistical Arbitrage - Homework 3.

This assignment is due by February 3rd, 2026 at 11:59pm Pacific Time and should be done in groups of 2-3 people. We recommend collaborating via a private GitHub repo.

Instructions for this assignment are below. **Please read all instructions carefully.**

1. Replace "Your Name(s) Here" above in bold with your name, and, if working in a group, your group members' names.
2. Read and run the instructions code in the Setup section. Make sure you understand what it does.
3. Answer the questions in the sections below. Make sure to limit code output to a reasonable length so that the resulting PDF you'll make from this notebook is readable. If the resulting PDF is not readable, keep your lines of code less than 80 characters.
4. Once finished, export or convert the notebook to PDF, using any method you like. For example, you can use the File -> Save and Export Notebook As -> PDF option in Jupyter Lab, you can use the nbconvert command line tool, etc. Whatever displays best is fine. Make sure no code is cut off in the PDF and that sheets are 8.5x11 inch dimensions or similar.
5. Submit the resulting PDF to the Gradescope assignment. If working in a group, only one person needs to submit and select all group members on Gradescope.
6. Zip and submit all files (your .ipynb file and any .py files we provided or you created) to the Canvas assignment. If working in a group, only one member of the group needs to submit to Canvas. Unlike Gradescope, no group members need to be selected on Canvas.

Setup {-}

System Requirements

First, ensure you have the proper system requirements:

1. We will only support Mac or Linux in this course. If you're on Windows, learn to use WSL 2.0 to run Linux on Windows, and then run these commands via WSL.
2. Ensure you have Python 3.10 and pip installed. You also need something to run this Jupyter notebook, whether that's Jupyter Notebook, Jupyter Lab, an IDE like VS Code, or Google Colab. We recommend collaborating via GitHub and using Jupyter Lab or VS Code to complete this notebook.

Jupyter Lab

To use Jupyter Lab, we'll need to set up the Python virtual environment and install the requirements. We've provided example instructions for doing so below for those using Jupyter Lab. Note that you may need to substitute `pip` with `pip3` depending on your installation. You can learn more by reading the venv docs and the ipykernel docs. See documentation for Jupyter Lab if needed.

```
# Navigate to the root directory containing your course files
cd /path/to/mse244

# Place the requirements.txt file in this directory if it isn't already there
# Put this .ipynb file in this directory or in a subdirectory, e.g. "hw1"

# Create a virtual environment
python3.10 -m venv .venv

# Activate the virtual environment
source .venv/bin/activate

# Install the required packages
pip install -r requirements.txt

# Add the virtual environment to Jupyter
python -m ipykernel install --user --name mse244 --display-name="MSE244"

# Start Jupyter Lab
jupyter lab

# Once in Jupyter, make sure to select the "MSE244" kernel
```

VS Code

If using VS Code, read the docs for Jupyter notebooks in VS Code and virtual environments in VX Code. VX Code can help you create and activate a virtual environment and Jupyter kernel for this notebook through its GUI (though you should also learn how to do these things via the command line). Make sure that you use Python 3.10 to create your virtual environment.

Once setup is complete and you're running within your environment, you can begin executing the cells below.

Imports {-}

In [31]:
```
%load_ext autoreload
%autoreload 2

import warnings
from datetime import datetime
from typing import Any, Dict, Optional, Tuple, Union

import arch.unitroot
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.ticker as mpl_ticker
import seaborn as sns
import scipy as sp
from scipy import stats
from statsmodels.graphics.tsaplots import plot_acf

# We add any local imports (Python files we've written) here
from factor_models import (
    load_data,
    select_asset_universe,
)


# Configure pandas display options
pd.set_option('display.max_columns', 20)
pd.set_option('display.max_rows', 40)
pd.set_option('display.precision', 4)
```

```
The autoreload extension is already loaded. To reload it, use:
  %reload_ext autoreload
```

Data {-}

To start with, we'll load the supplied Japanese equity data, preprocess the data, and set up the necessary variables.

Please make sure to look up and learn about any lines you don't understand by reading the Pandas User Guide. This is essential knowledge for anyone working with financial data in Python.

In [32]:
```
config = {
    'NIKKEI_CSV_PATH': 'N225.csv',
    'END_YEAR': 2020,
}

prices, returns, tickers, metadata = load_data(config, verbose=True)
```

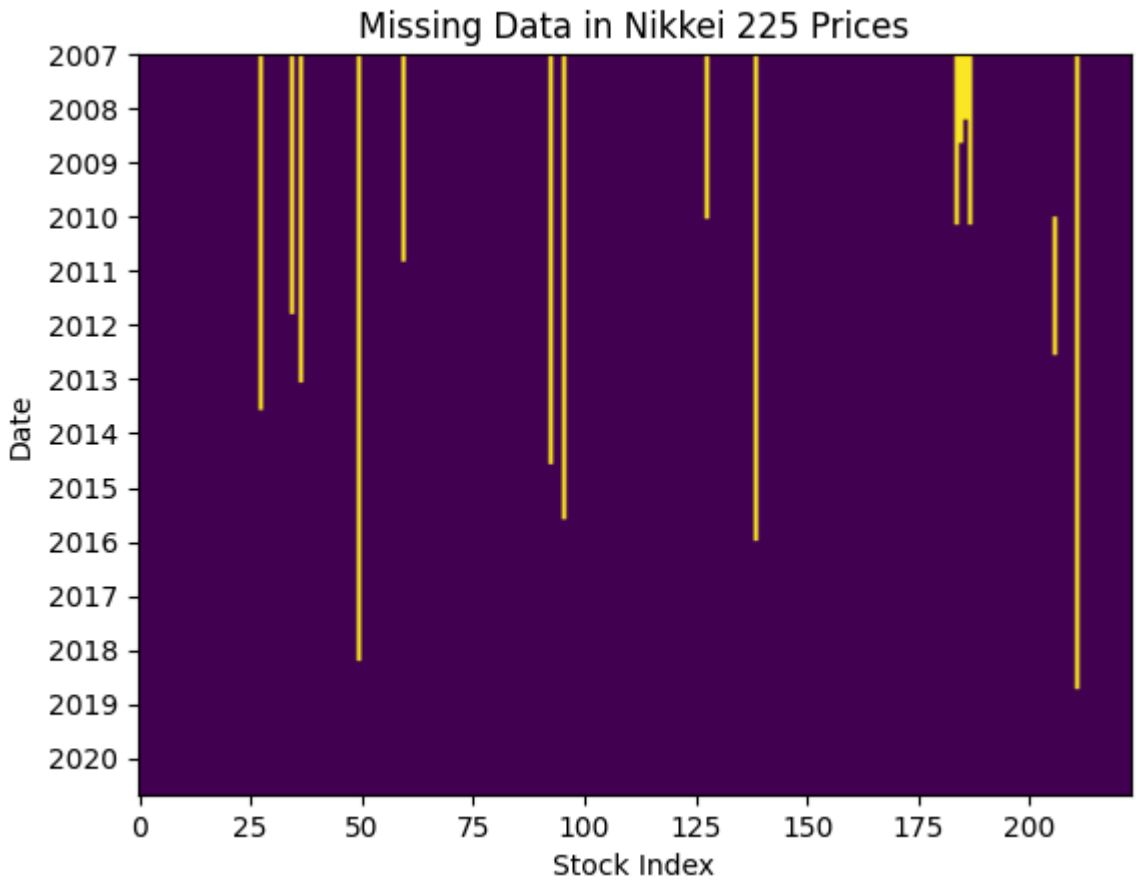
```
Price df shape at load: (4235, 226)
Price df shape after slicing time axis: (3447, 226)
Price df shape after removing future asset columns: (3447, 224)

Prices head:
```

Date	1332.T	1605.T	1721.T	1801.T	1802.T	1803.T	1808.T	1812.T	1925.T	1928.T	...	9503.T
2007-01-04	514.1772	1640.3337	906.6639	1236.6694	523.9589	456.3690	1560.3464	682.8918	1352.8394	1015.7311	...	2240.7510
2007-01-05	535.0902	1598.5864	909.3971	1226.7228	513.8568	450.3043	1542.5345	677.7186	1339.8625	1005.1937	...	2172.8496
2007-01-09	524.2733	1614.2418	925.7949	1256.5621	520.5914	460.9175	1538.9718	695.8253	1372.3043	1015.7311	...	2179.6396
2007-01-10	506.2446	1581.1916	913.4961	1246.6157	511.8364	466.2240	1535.4095	691.9454	1356.0836	1006.3643	...	2138.8989
2007-01-11	504.0811	1589.8885	910.7634	1249.9312	511.8364	479.8697	1567.4717	699.7054	1330.1299	1010.4626	...	2138.8989

5 rows × 223 columns

```
Metadata head:
```

Ticker	Industry	Sector	Company
1332.T	Fishery	Consumer Goods	NISSUI.CORP.
1605.T	Mining	Materials	INPEX.CORP.
1721.T	Construction	Capital Goods/Others	COMSYS.HOLDINGS.CORP.
1801.T	Construction	Capital Goods/Others	TAISEI.CORP.
1802.T	Construction	Capital Goods/Others	OBAYASHI.CORP.



Outline {-}

In this homework, we'll explore how factor models used in statistical arbitrage are defined, estimated, and analyzed. The lessons from this homework will be used in the next homework as part of the strategy from Avellaneda & Lee paper on statistical arbitrage. The analysis here will be on in-sample data, but we'll consider out-of-sample data in the next homework.

In the paper, the authors design, backtest, and analyze a statistical arbitrage strategy which generalizes pairs trading. The strategy is based on the idea that the statistical noises (i.e. the residuals) which remain after modeling a cross-section of stock prices with a factor model are on average mean-reverting and weakly dependent at most times.

In this homework, we'll focus on the types of factor models utilized in the paper. We'll learn about two types of factors: statistical factors which we'll use PCA to extract from stock prices, and sector factors (a type of fundamental factor) which we'll also extract from stock prices.

For both of these tasks, we'll use the stocks of Japanese equities in the Nikkei 225 index, which is similar to the S&P 500 for Japan. We'll also use a more modern time period than Avellaneda & Lee.

Here's an outline of what you'll do in this homework:

1. Establish a few short mathematical results for using fundamental factor models in statistical arbitrage.
2. Complete code to implement the estimation procedure for a sector factor model.
3. Establish a few short mathematical results about PCA.
4. Complete code to implement the estimation procedure for a statistical factor model using PCA.
5. Compute some factors and residuals for the sector and PCA factor models and analyze a few things.
6. Compare the residuals from the factor models and test a few things.

Throughout the homework, feel free to make any modifications to the code or models which you think would improve their out-of-sample behavior.

You can also use any additional data, packages, or tools which you think will be helpful.

Factor Model Review {-}

Introduction {-}

In Gatev et al., the authors form two-asset residuals by looking for pairs of assets which were close in normalized price. They do this to find residuals which are mean-reverting.

In Avellaneda & Lee, we will instead use a factor model to form multi-asset residuals. Using a factor model is a more general approach which allows us to form residuals for more than two assets without performing a combinatorial search. The factor model is a linear regression model that explains the returns of each asset as a linear combination of factors. The residuals are the difference between the actual returns and the returns predicted by the model. The factors are typically chosen to be common to all assets, such as the return of

all assets in the market, or the return on a portfolio of assets. The residuals are then traded on the assumption that they are mean-reverting.

In the following questions, you will write code to build a factor model for the Nikkei 225 stocks. You will then write code to form the residuals from the factor model. The residuals are the difference between the actual returns and the returns predicted by the model.

Definition {-}

We will use the following basic notation, with more defined below:

- $R \in \mathbb{R}^{T \times N}$ is the matrix of returns, where $R_{t,n}$ is the return of asset n at time t.
- $F \in \mathbb{R}^{T \times K}$ is the matrix of factor returns, where $F_{t,k}$ is the return of factor k at time t.
- $\beta \in \mathbb{R}^{T \times K \times N}$ is the tensor of factor betas (sometimes called factor loadings, or factor exposures), where $\beta_{t,k,n}$ is the factor beta of asset n to factor k at time t.
- $\alpha \in \mathbb{R}^{T \times N}$ is the matrix of alphas, where $\alpha_{t,n}$ is the alpha of asset n at time t.
- $\epsilon \in \mathbb{R}^{T \times N}$ is the matrix of residuals, where $\epsilon_{t,n}$ is the residual return of asset n at time t.
- T is the number of time periods.
- N is the number of assets.
- K is the number of factors.

The factor model posits a model for returns which is given by the following expression:

$$R_{t,n} = \alpha_{t,n} + F_t \beta_{t,n} + \epsilon_{t,n}$$

Here we will refer to $\alpha_{t,n} \in \mathbb{R}$ as the *intrinsic component*, $F_t \beta_{t,n} \in \mathbb{R}$ as the *systematic component* (with $F_t \in \mathbb{R}^{1 \times K}$ and $\beta_{t,n} \in \mathbb{R}^{K \times 1}$), and $\epsilon_{t,n} \in \mathbb{R}$ as the *idiosyncratic component*. The idiosyncratic component is sometimes referred to as the specific component or the idiosyncratic error. Various statistical assumptions can be made about each of these components, and about the loadings and factors as well. These assumptions result in various consequences for statistical inference performed with factor models, and there's a vast econometric literature which covers these topics. However, in this homework (and generally in this class as well), we will not concern ourselves with them, as factor models exist in stat arb primarily as a tool to build residuals.

Something to note is that we have written *time-varying* loadings in this definition: the term β has a t index. This is a generalization of the classic factor model definition which allows for the factor betas to change over time. This is a common feature of factor models in

practice, and the one which we will use in this homework. The static loadings case, which may be more familiar, is a special case of this model where the factor betas are constant over time.

You might also notice that this may be slightly different from the definition we've established in class: factor models are classically written with all of the variables we've defined here roughly transposed (and potentially with some axes permuted); this equation looks like $R_{n,t} = \alpha_{n,t} + \beta_{n,t}^\top F_t + \epsilon_{n,t}$ or $R_{n,t} = \alpha_{n,t} + \beta_{n,t} F_t + \epsilon_{n,t}$, depending on how one organizes the betas. However, this equation does not put the time axis as the first axis, which can be a bit unintuitive for those implementing these models in code for the first time. In implementation, some of these variables would generally be permuted or have their axes transposed. Some bugs are then invariably found when some axes are not aligned correctly, which we'd rather save you from experiencing.

Uses of Factor Models {-}

Factor models have a variety of applications in finance, across both academia and industry. These applications generally skew towards either performing statistical inference, or making predictions, though some cases blend the two.

Asset pricing tests. In academia, factor models have been used to test various asset pricing models. These tests put various economic or statistical assumptions on the meaning of the factors and their behavior and posit various relationships to hold in reality. In this literature, each of the components above (including the intercept, α) are generally studied and utilized in some way. Asset pricing tests are mostly not of interest for statistical arbitrage, so we won't expound on it. This is an inferential task.

Performance decomposition. In industry, factor models are very widely used to decompose the variation in returns of investment managers or funds, thereby explaining how much edge they have. In this use case, two types of factors (corresponding to the fundamental factors below) are generally used: style factors and industry factors. Style factors roughly proxy for well-known investment strategies, such as investing in stocks which have low prices as compared to the value of their business (the "value" factor) or investing in stocks which have increased in price recently (the "momentum" factor). Industry factors simply measure industry indices of performance. A general "market factor" capturing the performance of most stocks (e.g. a market cap-weighted index) is always used as well. In this application, α represents the skill of the investment manager which is unattributable to these well-known sources of variation in returns. This is the source of the term "alpha": if you have alpha, your α is positive and significant ("significant" here being a connection to the prior application, asset pricing tests). This sits in contrast to β, which roughly refers to the loadings on each of these well-known factors. For quantitative fund managers, large β is undesirable, because it opens up the strategy to risk from these factors. In fact, these factors are often referred to as "risk factors", because they generally are seen as risky to bet on. Being market-neutral

means having a low beta to the market factor. Being factor-neutral means having low beta to the other factors(or at least, the ones that matter to whoever you're talking to). We won't talk about this too much, except for analyzing the performance of our strategies. This is an inferential task.

Portfolio optimization. As we will see in portfolio optimization, portfolios can be optimized in order to minimize beta to the risk factors referred to in the paragraph above. We won't talk too much about this now, but we'll hear more about it in future classes. This is arguably a predictive task.

Covariance matrix estimation. Factor models are especially useful for covariance matrix estimation, as they reduce the number of parameters being estimated. A covariance matrix requires $O(N^2)$ parameters to be estimated. However, if the returns follow a factor model with static betas with $K = 5$ over some lookback window L, then we only need to estimate $O(LK + KN)$ parameters, which is often much smaller than $O(N^2)$. The estimated covariance matrices are often used in portfolio optimization. This can be an inferential or predictive task, depending on the application.

Return forecasting. Factor models are not only used to estimate returns; they're also used for return and volatility forecasting. In this application, factor models may follow the equation $R_{t,n} = \alpha_{t-1,n} + \beta_{t-1,n}^\top F_t + \epsilon_{t,n}$ in order to predict the future return instead of explaining the present return. This is generally a predictive task.

Statistical arbitrage. In stat arb, factor models are a tool to estimate residuals. We'll describe this application next. The type of task this is depends on one's view of the market, though perhaps it errs more towards the predictive than the inferential. Regardless, we'll use the traditional form of the factor model where all time subscripts are set to the same index (i.e. not the predictive equation) for our purposes in this homework. (Though you might want to try out this application with the predictive equation, and see if you get better results!)

Residuals {-}

Regardless of how the factors are computed, any factors can be regressed on returns at time t, $R_t \in \mathbb{R}^{1\times N}$, so that the factors are portfolios of assets which are given by the *factor weights* $W^F \in \mathbb{R}^{T\times N\times K}$ with $W_t^F \in \mathbb{R}^{N\times K}$:

$$F_t = R_t W_t^F \in \mathbb{R}^{1\times K}$$

Rearranging things, we see that the residuals $\epsilon_t \in \mathbb{R}^{1\times N}$ are given by:

$$\epsilon_{t,n} = R_{t,n} - F_t \beta_{t,n}$$

Here we note that $\alpha_{t,n}$ has been absorbed into the residual term $\epsilon_{t,n}$. This is a reasonable practice when using factor models for statistical arbitrage, as the alphas are often not of much interest in the context of creating residual portfolios. Moreover, we'll also model the mean of the residuals (potentially in a better, more dynamic way than using a constant intercept per asset) when we trade the residuals using some mean-reversion model after this residual estimation step.

In any case, under this definition, the residuals are (as desired) *traded* portfolios for the *composition matrix* $\Phi_t \in \mathbb{R}^{N \times N}$:

$$\epsilon_t = R_t - F_t \beta_t = R_t - R_t W_t^F \beta_t = R_t \underbrace{\left(I_N - W_t^F \beta_t\right)}_{\Phi_t}$$

Here I_N is the identity matrix in $\mathbb{R}^N$. The collection of composition matrices is the composition matrix set $\Phi \in \mathbb{R}^{T \times N_t \times N_t}$. Note that we have the subscript t on N in the composition matrix set's dimensions, since we may select a different asset universe at each time we form residuals, which will typically have a different number of assets.

The weights of each residual portfolio are given by this composition matrix. So, taking a position in the residual is the same as taking a position in a stock portfolio, where the portfolio weights are given by the composition matrix. To be specific, the i-th column of the composition matrix gives the weights (on each stock) for the i-th residual portfolio. If this doesn't make sense to you, try performing the matrix-vector multiplication $R_t \Phi_t$ for a simple 3×3 example and see what you get. And note that of course this depends on the way we arranged the returns and betas! Under some other shapes for the returns, betas, factors, etc., it may be the rows of the composition matrix which give the weights.

N.B.: the term "composition matrix" may not be standardized, and various authors may refer to this object with other names, e.g. transition matrix, projection matrix, etc.

Types of Factor Models {-}

In a factor model, the factors are typically chosen to be common to all assets; that is, they're used to explain the common variation in the returns of the assets. There are three types of factor models which are commonly used in practice. These three types of models roughly translate into different conventions on where the factors come from, and how they are estimated.

Macroeconomic Factor Models {-}

These factor models use factors which are typically extracted from macroeconomic data. These factors are often chosen to be interpretable, such as the return on the S&P 500, the return of the price of oil, the inflation rate, the unemployment rate, etc. These factors are often not *traded*, which means that they aren't tradeable portfolios of assets within our chosen asset universe (i.e. the N assets). For example, oil is not a Japanese equity, so it's not traded. As we will see, for stat arb, we want factors to be traded, so in this case, we'll need to perform an estimation procedure to regress the non-traded factor on the asset returns so that a close approximation (which is traded) is made. In macroeconomic factor models, the factors are thus given, and the betas need to be estimated. These betas are estimated via time-series regression (see the Estimation section for what this means).

Fundamental Factor Models {-}

These factor models use information which is extracted from the metadata (aka fundamentals) of the assets. Common sources include financial filings, the activities of the entity which issued the asset, or some economic information derived from the asset's price. Values representing this information are often chosen to be interpretable, such as the price-to-earnings ratio, the price-to-book ratio, the return on equity, or the sector of the stock. However, these values are also not traded. There are two common methods to deal with this:

Barra method. The first method treats these values as betas. Thus the betas are given and the factors need to be estimated. The factors are estimated by cross-sectional regressions (see the Estimation section for what this means). This is where the Barra method is generally considered to end. However, for stat arb, we need the factors to be traded portfolios. So, following estimation of the factors, they can be regressed on asset returns as in macroeconomic factor models. The betas can then be re-estimated. If necessary, this process can be repeated until convergence. We won't use this method in this homework, given the complexity of its implementation. It's generally associated with Barra, a company which provides factor models for risk estimation and portfolio optimization.

Fama(-French) method. The second method forms portfolios of assets using these values to define factor weights, and then uses the returns of those portfolios as the factors. These portfolios of assets are generally formed in one of two ways: (1) If the value is scalar, the assets are sorted by the value into buckets, where each bucket contains some percentage of the assets. A long-short portfolio is then constructed by going long the top bucket and short the bottom bucket. Popular bucketing schemes generally take the top and bottom $q\%$, where q is usually e.g. 10, 20, or 30. The weights are usually uniform on each long/short side. More complex schemes are also used. (2) If the value is categorical, then multiple portfolios are generally formed for each category with particular weights (often equal weights, though again, more complicated schemes are possible). This gives us factors. Thus the betas need to be found by estimating the factor model via time-series regression. This is one of the methods we will use in this homework. It's generally associated with

Eugene Fama and Kenneth French, two financial economists who are famous for their work on asset pricing. Fama and French posited in 1994 a well-known three-factor model which uses the returns of portfolios of small minus big market cap stocks (the "size" factor), high minus low book-to-market stocks (the "value" factor), and the return on the market (the "market factor"). Fama won a Nobel Prize in 2013, partially for his work on this model and associated methods.

Statistical Factors {-}

These factor models use only information from market data of the assets themselves. One example of statistical factors are those which are extracted using Principal Component Analysis (PCA). The factors are often uninterpretable but sometimes correspond to recognizable phenomena. One nice property of statistical factors is that the factor weights are constructed through the process of estimating the factors. This means that the factors are automatically tradeable portfolios of the assets. This is one of the methods we will use in this homework. In statistical factor models, neither the factors nor the betas are given: both are estimated from the returns. Other methods might include volume information or use trade information as well.°

Estimation {-}

As mentioned above, the type of estimation procedure used depends on the type of factor model. Here we will discuss the estimation procedures for the two types of factor models we will use in this homework.

Fundamental Factor Model Estimation {-}

In a fundamental factor model, we will use the Fama Method. Thus we will form portfolios of assets using the fundamental data. The factors are the returns of these portfolios. We then regress these portfolios on the returns of the assets to estimate the factor betas. The factor betas are the coefficients of the regression (as a reminder, these are often called the loadings of the assets on the factors).

The estimation is somewhat complicated by the fact that we have time-varying loadings. This is generally dealt with by using a *lookback window* to estimate β_t in a rolling fashion.

So, at each t, β_t is estimated through a *time-series regression*. A time-series regression is a regression where the dependent variable is a time series, and the independent variables are also time series. In this context, the independent variables must be stationary (for example, they might be returns). The factor betas are estimated by solving the following regression optimization problem for some lookback window L:

$$\min_{\beta_t} \|R_{t-L:t} - F_{t-L:t}\beta_t\|_2^2$$

Here the subscript notation $t - L : t$ denotes the last L observations of the time series up to but not including time t, so that $R_{t-L:t} \in \mathbb{R}^{L\times N}$ and $F_{t-L:t} \in \mathbb{R}^{L\times K}$, and of course $\beta_t \in \mathbb{R}^{K\times N}$. If it doesn't jump out at you immediately, to be clear, this is just simple OLS. This is a standard way to estimate this model, though more advanced approaches to regression can be used here as well (including methods of standardizing the factors). Note that we set $\alpha_t = 0$ (as we will throughout this homework), though it can be estimated. The factor betas are then used to form the residuals.

Statistical Factor Model Estimation {-}

In a statistical factor model, only the market data is observed, and we estimate both the factors and the loadings. We will assume that $T > N$ and use principal components analysis (PCA) for this task. If $T < N$, more advanced methods must be used, which we won't cover in this homework. The general recipe for this estimation is as follows:

1. First, we will extract factor weights. This is performed via eigendecomposition of the covariance matrix or correlation matrix of returns. This is the PCA step.
2. Next, we will compute the factors from the factor weights. The factors will thus be traded, and these portfolios are known as eigenportfolios.
3. Finally, we will regress the returns on the factors in a time series regression to estimate the loadings.

Specifically, we eigendecompose either the empirical covariance matrix, which is defined as

$$C = \frac{1}{T-1}(R - \bar{R})^\top (R - \bar{R}).$$

Here $\bar{R} \in \mathbb{R}^{T\times N}$ is the mean return matrix which contains in each row the time series mean of the returns of each asset. Note that the correlation matrix is also sometimes used, but we will not use it here.

PCA is a dimension reduction technique which forms N principal components from N asset time series. In PCA, the covariance matrix is decomposed as

$$C = V\Lambda V^\top$$

where $V \in \mathbb{R}^{N \times N}$ is the matrix of eigenvectors (where each column is an eigenvector) and $\Lambda \in \mathbb{R}^{N \times N}$ is the diagonal matrix of eigenvalues. The eigenvectors and eigenvalues are constructed and ordered such that the largest eigenvalue occurs first, the second-largest second, etc. The eigenvectors V are known as the *principal directions* or *principal axes* of the data. The projections of the data onto these axes RV are known as the *principal components*. The jth column of RV is thus the jth principal component. Note that the eigenvectors are constructed such that $V_i^\top V_j = 0$ for $i \neq j$ and $\|V_i\|_{L^2} = 1, \forall i$.

In the context of statistical arbitrage, we will only select the top K principal axes, thus reducing V to an $N \times K$ matrix and Λ to a $K \times K$ matrix. These top K principal axes are then used as the K factor weights (just as $R_t W_t^F = F_t$, we have $R_t V = F_t$), and the principal components are used as the factors. Thus the factors (also known as eigenportfolios) are traded, as they're linear combinations of the assets. Note here that the weights are sometimes scaled in some other way before constructing the factors, which we won't cover here.

Now that we have the factors, we estimate the betas. This is done as we've already outlined it above: the estimation is again complicated by the fact that we have time-varying loadings, and this is again generally dealt with by using a lookback window to estimate the factors F_t and the loadings β_t. In practice, these lookback windows may be different, with the window for the factors being usually longer than the window for the loadings.

Other Estimation Notes {-}

Here we'll review a few other estimation techniques used in factor modeling which are useful to know.

Cross-sectional Regression {-}

A cross-sectional regression is a regression where we consider just one time point at a time. The dependent variable is the returns of the assets at that time, and the independent variables are the factors or the betas at that time. This is used in the Barra method to estimate factors from given betas.

In the Barra method, the factors are estimated by solving the following regression optimization problem at each time period t:

$$\min_{F_t} \|R_t - F_t \beta_t\|_2^2.$$

Note that more advanced approaches, such as using a weighting matrix with different entries for each asset or enforcing various conditions on the factors such as orthogonality, are also used. Typically the values used for betas are also standardized by e.g. z-

scoring. We won't cover the Barra method in this homework.

Non-tradeable Factors {-}

In some cases, the factors are not tradeable. This is often the case in macroeconomic factor models. In this case, we need to estimate the factors from the returns of the assets. If the non-tradeable factors are denoted G_t, then we can find the factor weights W_t^F by solving the following regression optimization problem at each time period t:

$$\min_{W_t^F} \sum_{t=1}^{T} \left\| G_t - R_t W_t^F \right\|_2^2.$$

The projected factors used in the factor model are then $F_t := R_t W_t^F$.

Rolling Window Estimation {-}

In rolling window estimation, the factor model is estimated at each time point using a fixed lookback window. This is useful for estimating the time-varying loadings of the factor model. The lookback window is a hyperparameter which needs to be chosen.

Kalman Filters {-}

The Kalman filter is a recursive algorithm which estimates the state of a linear dynamic system from a series of noisy measurements. It can also be used to estimate the factor betas in a factor model. However, it requires some more work to implement, and we won't use it in this homework.

References {-}

1. Gatev, E., Goetzmann, W. N., & Rouwenhorst, K. G. (2006). Pairs trading: Performance of a relative-value arbitrage rule. The Review of Financial Studies, 19(3), 797-827.

2. Avellaneda, M., & Lee, J. H. (2010). Statistical arbitrage in the US equities market. Quantitative Finance, 10(7), 761-782.

3. Engle, R. F., & Granger, C. W. J. (1987). Co-integration and error correction: Representation, estimation, and testing. Econometrica, 55(2), 251-276. https://www.jstor.org/stable/1913236

4. Chen, N. F., Roll, R., & Ross, S. A. (1986). Economic Forces and the Stock Market. Journal of Business, 59(3), 383-403. https://pages.nes.ru/agoriaev/Papers/Chen%20APT%20macro%20factors%20JB86.pdf

5. Fama, E. F., & French, K. R. (1993). Common risk factors in the returns on stocks and bonds. Journal of Financial Economics, 33(1), 3-56.

6. Fama, E. F., & French, K. R. (2015). A five-factor asset pricing model. Journal of Financial Economics, 116(1), 1-22. https://papers.ssrn.com/sol3/papers.cfm?abstract_id=2287202

7. Doz, C., & Fuleky, P. (2019). Dynamic Factor Models. Technical Report halshs-02262202f. https://shs.hal.science/halshs-02262202

8. Rosenberg, B., & Rudd, A. (1982). Factor-Related and Specific Returns of Common Stocks: Serial Correlation and Market Inefficiency. The Journal of Finance, 37(2), 543-554. https://onlinelibrary.wiley.com/doi/abs/10.1111/j.1540-6261.1982.tb03575.x

9. Blin, J., Guerard, J., & Mark, A. (2022). A history of commercially available factor models. In: Cheng-Few Lee & Alice C. Lee (ed.), Encyclopedia of Finance, edition 0, chapter 97, pages 2275-2311, Springer. https://ideas.repec.org/h/spr/sprchp/978-3-030-91231-4_99.html

Questions {-}

Question 1

(2 points, 1 point per part)

Part 1

Prove that the composition matrix $\Phi_t \in \mathbb{R}^{N \times N}$ can be used to take a vector of positions in the residuals $P_t^R \in \mathbb{R}^N$ and transform it to a vector of positions in the assets $P_t^A \in \mathbb{R}^N$.

This should only take a few lines.

Prove that the composition matrix $\Phi_t \in \mathbb{R}^{N \times N}$ can be used to take a vector of positions in the residuals $P_t^R \in \mathbb{R}^N$ and transform it to a vector of positions in the assets $P_t^A \in \mathbb{R}^N$.

This should only take a few lines.

Solution:

The composition matrix $\Phi_t \in \mathbb{R}^{N \times N}$ is defined $\Phi_t = I_N - W_t^F \beta_t$

The residual position represents the allocation to assets based on its residuals i.e. w.r. the factor model

$$R_t^A = W_t^F F_t + \epsilon_t$$

The residual positions should be allocated s.t. they have no factor exposure. Hence we just need to show that applying β_t to $\Phi_t P_t^R = 0$. We have:

$$\beta_t P_t^A = \beta_t (I_t - W_t^F \beta_t) P_t^R = \beta_t P_t^R - \beta_t W_t^F \beta_t P_t^R = \beta_t P_t^R - I_K \beta_t P_t^R = \beta_t P_t^R - \beta_t P_t^R = 0$$

I.e. the assets positions P_t^A have no factor exposure. Hence $P_R^A = \Phi_t P_t^R$, dimensions do match $(1 \times N)$. To clarify, since factor portfolios reproducde their exposure i.e.

$$\beta_t W_t^F F_t = F_t \Leftrightarrow \beta_t W_t^F F_t F_t^{-1} = F_t F_t^{-1} \Leftrightarrow \beta_t W_t^F = I_K$$

Part 2

Give the solution to the following optimization problem from the fundamental factor model time-series regression, when the Fama-French Method has been used to construct factors:

$$\min_{\beta_t} \|R_{t-L-1:t} - F_{t-L-1:t} \beta_t\|_2^2.$$

You may cite any results which are taught in a standard course on linear algebra, probability, statistics, etc. This should only take a line or two.

Solution:

$$\min_{\beta_t} \|R_{t-L-1:t} - F_{t-L-1:t}\beta_t\|_2^2$$

We take the derivative

$$\begin{aligned}
&\frac{d}{d\beta_t}(R_{t-L-1:t} - F_{t-L-1:t}\beta_t)^T(R_{t-L-1:t} - F_{t-L-1:t}\beta_t) \\
&= \frac{d}{d\beta_t} R_{t-L-1:t}^T R_{t-L-1:t} - R_{t-L-1:t}^T F_{t-L-1:t}\beta_t - (F_{t-L-1:t}\beta_t)^T R_{t-L-1:t} + (F_{t-L-1:t}\beta_t)^T(F_{t-L-1:t}\beta_t) \\
&= \frac{d}{d\beta_t} R_{t-L-1:t}^T R_{t-L-1:t} - R_{t-L-1:t}^T F_{t-L-1:t}\beta_t - \beta_t^T F_{t-L-1:t}^T R_{t-L-1:t} + \beta_t^T F_{t-L-1:t}^T F_{t-L-1:t}\beta_t \\
&= \frac{d}{d\beta_t} R_{t-L-1:t}^T R_{t-L-1:t} - 2\beta_t^T F_{t-L-1:t}^T R_{t-L-1:t} + \beta_t^T F_{t-L-1:t}^T F_{t-L-1:t}\beta_t \\
&= -2F_{t-L-1:t}^T R_{t-L-1:t} + 2F_{t-L-1:t}^T F_{t-L-1:t}\beta_t \\
&\Leftrightarrow \beta_t = (F_{t-L-1:t}^T F_{t-L-1:t})^{-1} F_{t-L-1:t}^T R_{t-L-1:t}
\end{aligned}$$

Question 2

(4 points)

We will now implement a fundamental factor model which uses the Fama method to form the factors and estimate the betas. The factors will be defined as the returns of equally-weighted portfolios of all of the assets in each sector. Complete the class below which builds the sector factor model and residuals from the returns and the metadata. Use vectorized operations where possible and try to avoid loops.

In your code, make sure to make the following variables properties of the class:

- `self.factor_weights` : the factor weights, defined as W_t^F in the review.
- `factors` : the factors, defined as F_t in the review.
- `self.betas` : the betas, defined as β_t in the review.
- `self.rhat` : the estimated returns, defined as $F_t\beta_t$ in the review.
- `self.comp_mtx` : the composition matrix, defined as Φ_t in the review.
- `self.residuals` : the residuals, defined as ϵ_t in the review.

Most of the work is done in the `fit` function, where we'll use time-series regression to estimate the betas. The `predict` function will apply the estimated betas and factor weights to the new data. In general, the process carried out in the `fit` function is the following:

1. Compute the factor weights.
2. Compute the factors.
3. Perform a time-series regression (i.e. regress the returns on the factors to get the betas).
4. Compute the composition matrix from the formula in the review.
5. Compute the estimated returns.
6. Compute the residuals.

There are a few things to note:

- The `fit` function is meant to be called on a window of returns in a rolling manner, where the returns end at time t. So, the `fit` function computes and stores the parameter estimates for β_t, Φ_t, etc. for a particular t. We assume that the asset universe has been selected before the `fit` function is called, so no need to do any selection of assets within this function.
- The `predict` function should just apply the estimated betas and factor weights to the new data. Specifically it will apply the factor weights to the given returns to get the factors, then apply the betas to the factors to get the predictions, and then form the residuals by subtracting the predictions from the given returns. In this homework, we'll just fit and predict on the same data for illustrative purposes (though this is not realistic in practice, obviously).
- For the sector factor model, W_t^F is easy to compute: for each sector s factor, the weight is just $1/N_s$ for each stock in the sector, and 0 for stocks outside the sector, where N_s is the number of stocks in the sector. As a side effect, this means that the sector factors are already tradeable portfolios of the assets by construction. There are six sectors and thus six factors.

If you'd like, you can implement beta estimation with the intercept term. If you do this, make sure to make `self.alphas` a property of the class.

In [33]:
```
import statsmodels.api as sm

class NikkeiSectorFactorModel():
    def __init__(
        self,
        metadata: pd.DataFrame,
        intercept: bool = False,
    ):
        """
        Initialize the Nikkei sector factor model.

        Parameters:
        - metadata (pd.DataFrame): An Nx3 matrix containing the metadata
          with columns [Industry, Sector, Company].
        - intercept (bool): Perform the estimation with an intercept term.
        """
        self.metadata = metadata
        self.intercept = intercept
        self.is_fit = False

        self.sectors = metadata['Sector'].unique()
        self.sectors.sort()
        self.sector_counts = self.metadata['Sector'].value_counts().sort_index()
        self.num_factors = len(self.sectors)

        # ADDED
        self.factor_weights = None
        self.betas = None
        self.rhat = None
        self.comp_mtx = None
        self.residuals = None
        self.alphas = None
        self.factors = None
        assert(self.num_factors > 0)

    def __compute_factors(self, returns):
        # First we assign each stock a sector using the index metadata which is the ticker
```

```python
        # metadata index must match returns columns (tickers)
        sector_map = self.metadata['Sector']
        dict_map = dict(zip(self.sectors, range(self.num_factors)))
        # group columns (stocks) by sector and average across columns
        factors = returns.groupby(sector_map, axis=1).mean()

        return factors


    def fit(self, returns: pd.DataFrame) -> Tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame]:
        """
        Fit the factor model to the given returns data and return the
        residuals, estimated returns, and composition matrix.

        Parameters:
        - returns (pd.DataFrame): A TxN matrix containing the returns data.

        Returns:
        - (pd.DataFrame) a TxN matrix of residuals with proper index/columns.
        - (pd.DataFrame) a TxN matrix of estimated returns with proper index/columns.
        - (pd.DataFrame) an NxN composition matrix with proper index/columns.
        """
        self.returns = returns
        self.T, self.N = returns.shape
        assert(self.num_factors < self.N)
        assert(self.N < self.T)

        # === YOUR CODE BELOW ===

        factors = self.__compute_factors(returns).values
        self.factors = factors

        # Prepare Factors Data
        if self.intercept:
            factors_data = sm.add_constant(factors)
        else:
            factors_data = factors

        # Fit OLS
        # params is (K+1 x N) if intercept=True, else (K x N)
        params = sm.OLS(returns, factors_data).fit().params
```

```
        # Conditionally extract Alphas and Betas
        if self.intercept:
            self.alphas = params.iloc[0, :]
            self.betas = params.iloc[1:, :]
        else:
            self.alphas = pd.Series(0.0, index=returns.columns)
            self.betas = params

        # Compute Rhat (Estimated Returns)
        # Note: self.betas is (K x N), factors is (T x K)
        rhat_values = factors @ self.betas.values
        if self.intercept:
            rhat_values = rhat_values + self.alphas.values

        self.rhat = pd.DataFrame(
            rhat_values,
            index=returns.index,
            columns=returns.columns
        )

        # Compute Residuals
        self.residuals = returns - self.rhat

        # Labels
        self.residuals.index = returns.index
        self.residuals.columns = returns.columns


        # Composition matrix
        b = self.betas.values
        W = b.T @ np.linalg.inv(b @ b.T)
        self.comp_mtx_df = np.eye(self.N) - W @ self.betas

        rhat_df = self.rhat
        residuals_df = self.residuals
        comp_mtx_df = self.comp_mtx_df
        # === YOUR CODE ABOVE ===

        self.is_fit = True
        return (residuals_df, rhat_df, comp_mtx_df)
```

```python
    def predict(self, returns: pd.DataFrame) -> Tuple[pd.DataFrame, pd.DataFrame]:
        """
        Estimate the out-of-sample residuals and factors using from the
        estimated factor model given the new returns.

        Parameters:
        - returns (pd.DataFrame): A T2xN matrix containing the new returns data.

        Returns:
        - (pd.DataFrame) a T2xN matrix of residuals with proper index/columns.
        - (pd.DataFrame) an T2xK matrix of factors with proper index/columns.
        """
        if not self.is_fit:
            raise Exception("Must call fit() on model first.")
        # === YOUR CODE BELOW ===

        factors = self.__compute_factors(returns)
        predictions = factors @ self.betas

        if self.intercept:
            # Add the alpha (intercept) calculated during fit
            predictions = predictions + self.alphas

        residuals_df = returns - predictions
        factors_df = factors


        # === YOUR CODE ABOVE ===
        return (residuals_df, factors_df)


# Testing
if __name__ == "__main__":
    sector_factor_model = NikkeiSectorFactorModel(metadata, intercept=True)
    residuals_df, rhat_df, comp_mtx_df = sector_factor_model.fit(returns)
```

```
/var/folders/3t/5vvh8l5x48s7tyvbx4b6v9xr0000gn/T/ipykernel_48820/3533843098.py:42: FutureWarning: DataFrame.groupby
with axis=1 is deprecated. Do `frame.T.groupby(...)` without axis instead.
  factors = returns.groupby(sector_map, axis=1).mean()
```

Question 3

(part 1: 2 points, part 2: 1 point extra credit)

PCA is often posed as two different problems: finding projections of the data which minimize the reconstruction error, and finding projections of the data which maximize the variance of the projected data. The first is perhaps the more natural interpretation for statistical arbitrage, but in this question, you'll show that these two problems are the same, and then you'll solve one of them for the first eigenvector.

To start with, assume our data is mean-centered, e.g. $R := R - \bar{R}$. Let V_1 be an arbitrary unit vector. We want to minimize the reconstruction error of R after projection onto V_1, that is, we want to minimize the squared distance of our projected data from our original data.

To make this concrete, we need to set up some additional notation. Consider some arbitrary asset in our universe, R_n. We'll denote the projection of R_n onto V_1 as P_n:

$$P_n := \frac{R_n^\top V_1}{V_1^\top V_1} V_1.$$

Note that since V_1 is a unit vector, we may drop the denominator in the expression above, as it's just equal to one.

Now we'll move onto the questions.

Part 1

(2 points)

According to the reconstruction interpretation of PCA, we want to find the optimal V_1^* such that the squared difference between the projection P_n and our original asset R_n is minimized. This suggests the following problem:

$$V_1^* = \underset{V_1}{\operatorname{argmin}} \sum_{n=1}^{N} \|P_n - R_n\|_2^2 \quad \text{s.t. } \|V_1\|_2 = 1$$

Show that this problem is equivalent to the following problem:

$$V_1^* = \underset{V_1}{\operatorname{argmax}} \sum_{n=1}^{N} \left(V_1^\top R_n\right)^2 \quad \text{s.t. } \|V_1\|_2 = 1$$

Solution:

Since $\|V_1\|_2 = 1$ we can rewrite the projection as

$$P_n = \frac{R_n^T V_1}{\|V_1\|_2} V_1 = (R_n^T V_1) V_1$$

Rewriting our objective function as

$$
\begin{aligned}
V_1^* &= \underset{V_1}{\operatorname{argmin}} \sum_{n=1}^{N} |P_n - R_n|_2^2 \\
&= \underset{V_1}{\operatorname{argmin}} \sum_{n=1}^{N} \left|(R_n^\top V_1)V_1 - R_n\right|_2^2 \\
&= \underset{V_1}{\operatorname{argmin}} \sum_{n=1}^{N} \left[((R_n^\top V_1)V_1 - R_n)^\top ((R_n^\top V_1)V_1 - R_n)\right] \\
&= \underset{V_1}{\operatorname{argmin}} \sum_{n=1}^{N} \left[((R_n^\top V_1)V_1)^\top ((R_n^\top V_1)V_1) - 2R_n^\top ((R_n^\top V_1)V_1) + R_n^\top R_n\right] \\
&= \underset{V_1}{\operatorname{argmin}} \sum_{n=1}^{N} \left[(R_n^\top V_1)^2 (V_1^\top V_1) - 2(R_n^\top V_1)(R_n^\top V_1) + |R_n|_2^2\right] \\
&\quad \text{Using the constraint } V_1^\top V_1 = 1: \\
&= \underset{V_1}{\operatorname{argmin}} \sum_{n=1}^{N} \left[(R_n^\top V_1)^2 - 2(R_n^\top V_1)^2 + |R_n|_2^2\right] \\
&= \underset{V_1}{\operatorname{argmin}} \sum_{n=1}^{N} \left[-(R_n^\top V_1)^2 + |R_n|_2^2\right] \\
&\quad \text{Since } |R_n|_2^2 \text{ is constant w.r.t. } V_1 \text{ and minimizing a negative is maximizing a positive:} \\
&= \underset{V_1}{\operatorname{argmax}} \sum_{n=1}^{N} (R_n^\top V_1)^2
\end{aligned}
$$

Part 2

(1 point extra credit)

In this part, we'll solve the problem identified above. The objective is to find the optimal V_1^* such that the squared length of P_n is maximized. The squared length of P_n is

$$\|P_n\|_2 = \left\| \frac{R_n^\top V_1}{V_1^\top V_1} V_1 \right\|_2 = \frac{|R_n^\top V_1| \|V_1\|_2}{V_1^\top V_1} = |R_n^\top V_1|$$

Thus we have the following optimization problem:

$$V_1^* = \underset{V_1}{\operatorname{argmax}} \sum_{n=1}^{N} \left(V_1^\top R_n\right)^2 \quad \text{s.t. } \|V_1\|_2 = 1$$

Solve this problem.

Solution:

This is an optimization problem, let us rewrite it in that form

$$\max_{V_1} \quad \sum_{n=1}^{N} (V_1^\top R_n)^2 \text{ s.t.} \qquad |V_1|_2 = 1$$
$$V_1 \in \mathbb{R}^D$$

Rewriting the objective function and solving using Lagrange mutlipliers

$$(V_1^\top R_n)^2 = (V_1^\top R_n)^T (V_1^\top R_n) = V_1^T R_n R_n^T V_1$$

Then we have

$$V_1^T \left(\sum_{n=1}^{N} R_n R_n^T \right) V_1 = V_1^T X V_1$$

We define the Lagrangian $\mathcal{L}$ with multiplier λ to enforce the constraint $V_1^\top V_1 = 1$:

$$\mathcal{L}(V_1, \lambda) = V_1^\top X V_1 - \lambda (V_1^\top V_1 - 1)$$

Taking the derivative with respect to V_1 and setting it to zero:

$$\nabla_{V_1}\mathcal{L} = 2XV_1 - 2\lambda V_1 = 0$$
$$XV_1 = \lambda V_1$$

This is the eigenvalue equation. To maximize the objective $V_1^\top X V_1$, we left-multiply by $V_1^\top$:

$$V_1^\top X V_1 = V_1^\top(\lambda V_1) = \lambda(V_1^\top V_1) = \lambda$$

Conclusion: To maximize the sum of squared projections, V_1^* must be the eigenvector of the matrix X corresponding to the largest eigenvalue $\lambda_{\max}$.

Question 4

(4 points)

We will now implement a statistical factor model which uses PCA to form the factors, and regression to estimate the betas. Complete the class below which builds the PCA factor model and residuals from the returns. Use vectorized operations where possible and try to avoid loops.

In your code, make sure to make the following variables properties of the class:

- `self.factor_weights` : the factor weights, defined as W_t^F in the review.
- `self.factors` : the factors, defined as F_t in the review.
- `self.betas` : the betas, defined as β_t in the review.
- `self.rhat` : the estimated returns, defined as $F_t\beta_t$ in the review.
- `self.comp_mtx` : the composition matrix, defined as Φ_t in the review.
- `self.residuals` : the residuals, defined as ϵ_t in the review.

Most of the work is done in the `fit` function, where we'll use PCA to estimate the factor weights and factors, and then perform time-series regression to estimate the betas. The `predict` function will apply the estimated betas and factor weights to the new data. In general, the process carried out in the `fit` function is the following:

1. Compute the factor weights and factors with PCA.

2. Perform a time-series regression (i.e. regress the returns on the factors to get the betas).
3. Compute the composition matrix from the formula in the review.
4. Compute the estimated returns.
5. Compute the residuals.

There are a few things to note:

- The `fit` function is meant to be called on a window of returns in a rolling manner, where the returns end at time t. So, the `fit` function computes and stores the parameter estimates for β_t, Φ_t, etc. for a particular t. We assume that the asset universe has been selected before the `fit` function is called, so no need to do any selection of assets within this function.
- The `predict` function should just apply the estimated betas and factor weights to the new data. Specifically it will apply the factor weights to the given returns to get the factors, then apply the betas to the factors to get the predictions, and then form the residuals by subtracting the predictions from the given returns. In this homework, we'll just fit and predict on the same data for illustrative purposes (though this is not realistic in practice, obviously).
- For the PCA factor model, W_t^F is the loadings matrix of the PCA, and F_t is the principal components of the returns.

If you'd like, you can implement beta estimation with the intercept term. If you do this, make sure to make `self.alphas` a property of the class.

In [62]:
```
# import pca
from sklearn.decomposition import PCA

class NikkeiPCAFactorModel():
    def __init__(
        self,
        num_factors: int = 6,
        intercept: bool = False,
    ):
        """
        Initialize the Nikkei PCA factor model with the given parameters.

        Parameters:
        - num_factors (int): The number of factors to estimate.
        - intercept (bool): Perform the estimation with an intercept term.
        """
        assert(num_factors > 0)
```

```
        self.num_factors = num_factors
        self.intercept = intercept
        self.is_fit = False

        # OUR CODE
        self.factor_weights = None
        self.factors = None
        self.betas = None
        self.alphas = None
        self.rhat = None
        self.comp_mtx = None
        self.residuals = None
        self.pca_fit = None

    def __compute_factors(self, returns):
        T = returns.shape[0]
        C = returns - returns.mean(axis = 0)
        sigma = ((C.T @ C) / (T - 1)).to_numpy()

        C = C.to_numpy()

        evals, evecs = np.linalg.eigh(sigma)
        i = np.argsort(evals)[::-1]
        evals = evals[i]
        evecs = evecs[:, i]


        nf = self.num_factors
        weights = evecs[:,:nf]

        rw = weights.sum(axis = 0, keepdims=True)
        weights = weights / rw

        factors_mult = returns.to_numpy() @ weights
        factors = factors_mult[:, : nf]

        self.factor_weights = pd.DataFrame(weights, index = returns.columns, columns = [f"Factor_{i}" for i in rang
        self.factors = pd.DataFrame(factors, index = returns.index, columns = [f"Factor_{i}" for i in range(nf)])
        return weights, factors
```

```python
    def fit(self, returns: pd.DataFrame) -> Tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame]:
        """
        Fit the factor model to the given returns data and return the
        residuals, estimated returns, and composition matrix.

        Parameters:
        - returns (pd.DataFrame): A TxN matrix containing the returns data.

        Returns:
        - (pd.DataFrame) a TxN matrix of residuals with proper index/columns.
        - (pd.DataFrame) a TxN matrix of estimated returns with proper index/columns.
        - (pd.DataFrame) an NxN composition matrix with proper index/columns.
        """
        self.returns = returns
        self.T, self.N = returns.shape
        assert(self.num_factors < self.N)
        assert(self.N < self.T)

        # === YOUR CODE BELOW ===
        returns = returns.dropna()

        self.__compute_factors(returns)

        if self.intercept:
            X = sm.add_constant(self.factors)
            Y = returns
        else:
            X = self.factors
            Y = returns

        model = sm.OLS(Y, X).fit()
        p = model.params

        if self.intercept:
            self.alphas = p.iloc[0,:]
            self.betas = p.iloc[1:,:]
        else:
            self.alphas = pd.Series(0.0, index = returns.columns)
            self.betas = p

        if self.intercept:
```

```
            fitted_values = self.alphas.values + self.factors.values @ self.betas.values
        else:
            fitted_values = self.factors.values @ self.betas.values

        res = returns.values - fitted_values

        self.rhat = pd.DataFrame(fitted_values, index = returns.index, columns = returns.columns)
        self.residuals = pd.DataFrame(res, index = returns.index, columns = returns.columns)

        weight_vals = self.factor_weights.values
        betas = self.betas.values

        residual_proj = np.eye(self.N) - (weight_vals @ betas)

        residuals_df = self.residuals
        rhat_df = self.rhat
        comp_mtx_df = pd.DataFrame(residual_proj, index = returns.columns, columns = returns.columns)
        self.comp_mtx = comp_mtx_df
        self.mu = returns.mean(axis = 0)

        # === YOUR CODE ABOVE ===

        self.is_fit = True
        return (residuals_df, rhat_df, comp_mtx_df)

    def predict(self, returns: pd.DataFrame) -> Tuple[pd.DataFrame, pd.DataFrame]:
        """
        Estimate the out-of-sample residuals and factors using from the
        estimated factor model given the new returns.

        Parameters:
        - returns (pd.DataFrame): A T2xN matrix containing the new returns data.

        Returns:
        - (pd.DataFrame) a T2xN matrix of residuals with proper index/columns.
        - (pd.DataFrame) an T2xK matrix of factors with proper index/columns.
        """
        if not self.is_fit:
            raise Exception("Must call fit() on model first.")
        # === YOUR CODE BELOW ===
```

```
        returns = returns.dropna()

        factors = returns.values @ self.factor_weights.values
        rhat_center = factors @ self.betas.values
        if self.intercept:
            rhat = self.alphas.values + rhat_center
        else:
            rhat = rhat_center

        res = returns.values - rhat
        residuals_df = pd.DataFrame(res, index = returns.index, columns = returns.columns)
        factors_df = pd.DataFrame(factors, index = returns.index, columns = [f"Factor_{i}" for i in range(self.num_

        # === YOUR CODE ABOVE ===
        return (residuals_df, factors_df)


# Testing
if __name__ == "__main__":
    sector_factor_model = NikkeiPCAFactorModel(intercept=False)
    residuals_df, rhat_df, comp_mtx_df = sector_factor_model.fit(returns)
```

Question 5

(4 points, 2 per part)

Now that you've completed these classes, we'll run some diagnostics and learn a few things about the estimated residuals.

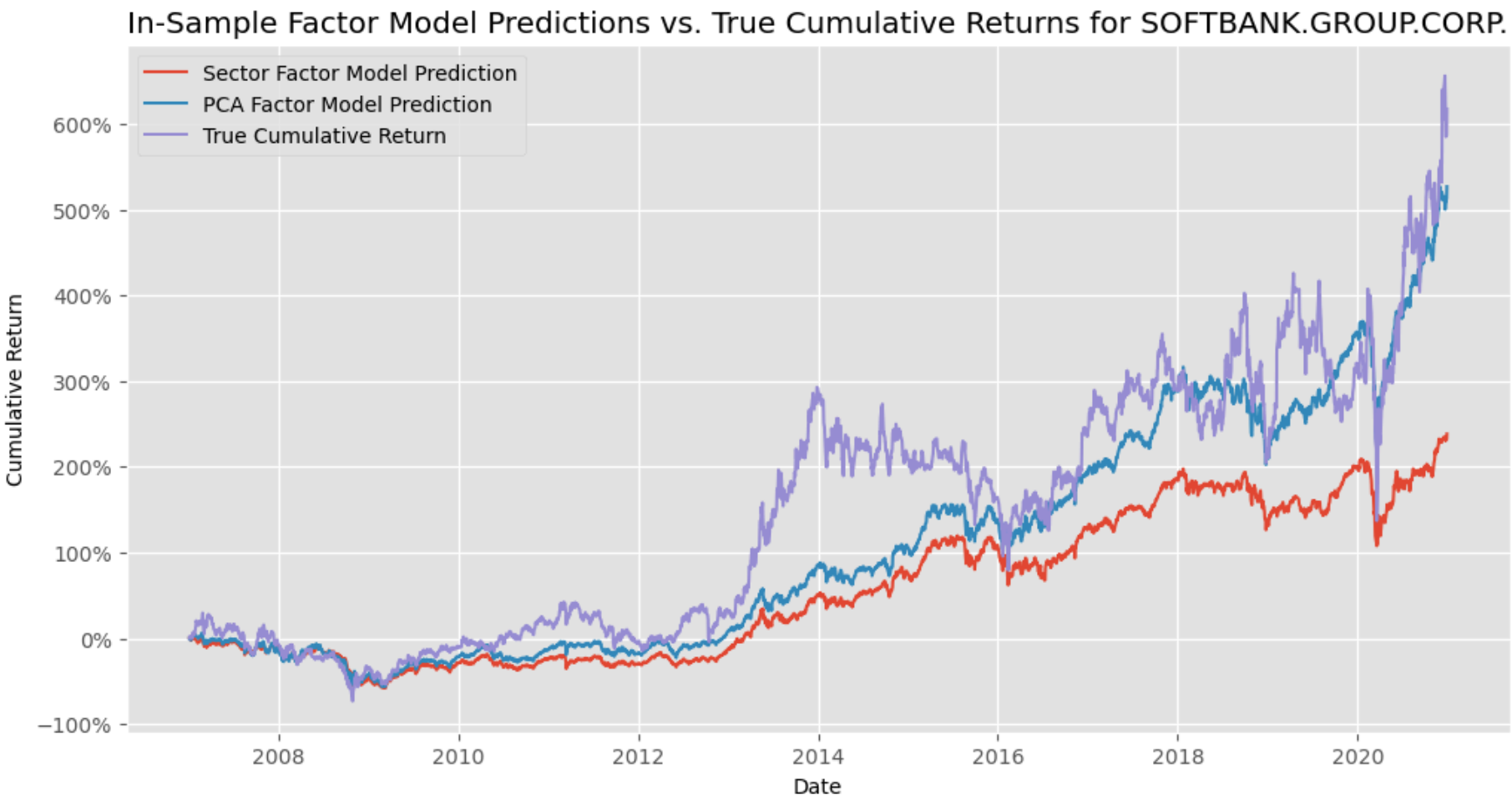
Run the following code block to output some information about your factor model. Feel free to modify any calls to your classes above as necessary.

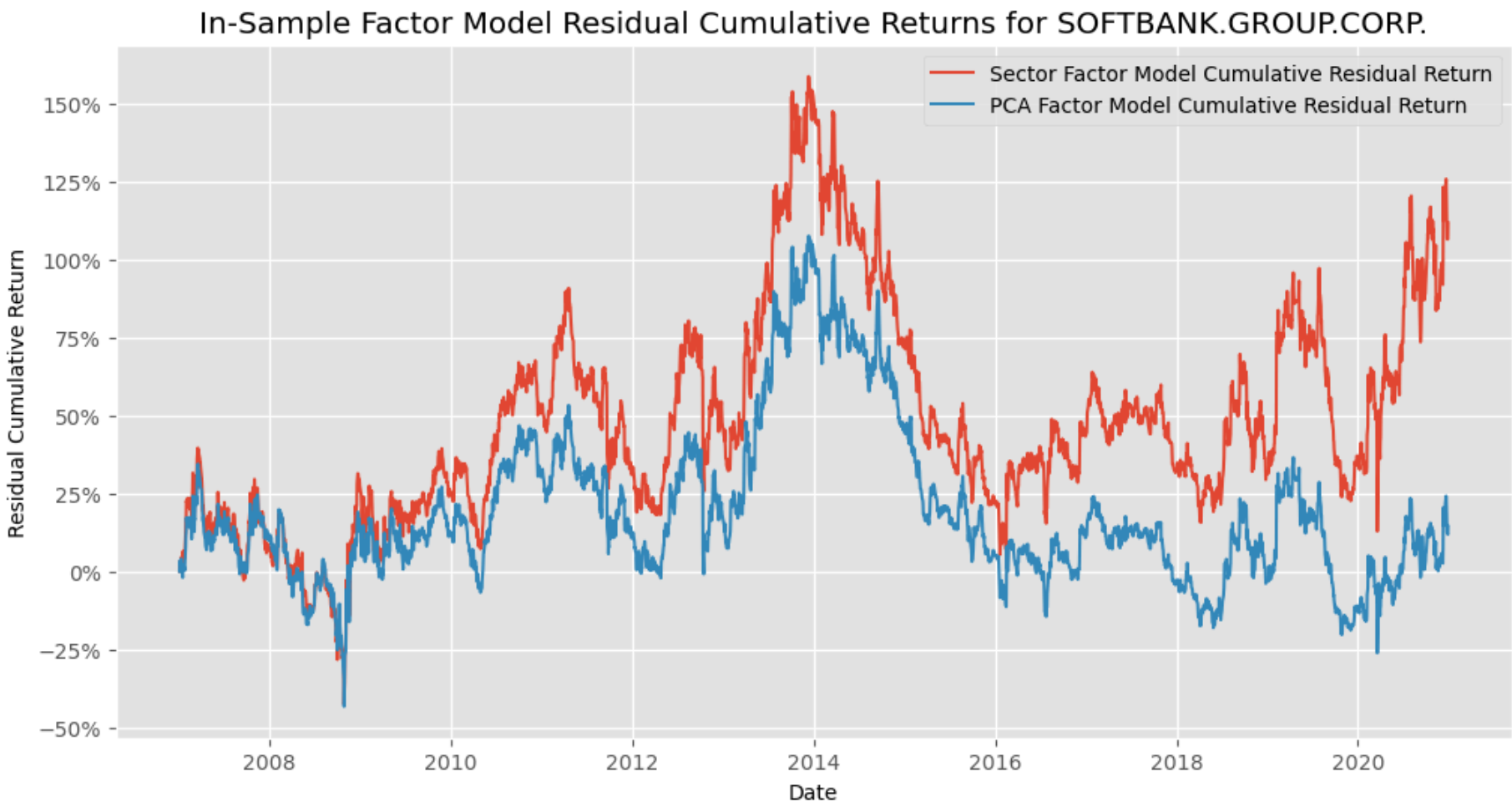
In [63]:
```
use_intercept = False

# Select the universe of assets to be used in the factor models
config = {
    'LOOKBACK_PERIOD': len(returns) - 1,
    'FILTER_MAX_ABS_RETURN': 0.5
```

```
}

valid_prices, valid_returns, valid_stocks = select_asset_universe(prices, returns, prices.index[-1], config)

# Fit the industry factor model and get residuals
sector_factor_model = NikkeiSectorFactorModel(metadata, intercept=use_intercept)
sector_residuals, sector_predictions, sector_comp_mtx = sector_factor_model.fit(valid_returns)
print(sector_residuals.shape, valid_returns.shape, valid_stocks.shape)
# Fit the PCA factor model and get residuals
pca_factor_model = NikkeiPCAFactorModel(num_factors=6, intercept=use_intercept)
pca_residuals, pca_predictions, pca_comp_mtx = pca_factor_model.fit(valid_returns)
print(pca_residuals.shape, valid_returns.shape, valid_stocks.shape)
# Verify the correctness of the composition matrices
sector_comp_mtx_error = np.abs(valid_returns.values @ sector_comp_mtx - sector_residuals.values).sum().sum()
print(f"Sector composition matrix error (should be less than 1e-10): {sector_comp_mtx_error:0.2g}")
pca_comp_mtx_error = np.abs(valid_returns.values @ pca_comp_mtx - pca_residuals.values).sum().sum()
print(f"PCA composition matrix error (should be less than 1e-10): {pca_comp_mtx_error:0.2g}")

# Print MAE of predictions for each model
print(f'Sector Factor Model MAE: {(np.abs(valid_returns - sector_predictions) * (valid_returns != 0)).mean().mean()
print(f'PCA Factor Model MAE: {(np.abs(valid_returns - pca_predictions) * (valid_returns != 0)).mean().mean():0.6f}

# Choose some asset to plot
ticker_to_plot = tickers[-1]  # softbank
company_name = metadata.loc[ticker_to_plot, 'Company']

# Plot the factor model predictions vs the actual cumulative returns for the ticker
print(sector_predictions.shape)
print(pca_predictions.shape)
print(valid_returns.shape)
print(returns.shape)
print(sector_predictions[ticker_to_plot].shape, pca_predictions[ticker_to_plot].shape, valid_returns[ticker_to_plot
with plt.style.context('ggplot'):
    fig, ax = plt.subplots(figsize=(12, 6))
    ax.plot((1 + sector_predictions[ticker_to_plot]).cumprod() - 1, label='Sector Factor Model Prediction')
    ax.plot((1 + pca_predictions[ticker_to_plot]).cumprod() - 1, label='PCA Factor Model Prediction')
    ax.plot((1 + valid_returns[ticker_to_plot]).cumprod() - 1, label='True Cumulative Return')
    plt.xlabel('Date')
    plt.ylabel('Cumulative Return')
    plt.title(f'In-Sample Factor Model Predictions vs. True Cumulative Returns for {company_name}')
    plt.legend()
```

```
    ax.yaxis.set_major_formatter(mpl_ticker.PercentFormatter(xmax=1))
    plt.show()

# Now plot the residuals of the factor models for the ticker
with plt.style.context('ggplot'):
    fig, ax = plt.subplots(figsize=(12, 6))
    ax.plot((1 + sector_residuals[ticker_to_plot]).cumprod() - 1, label='Sector Factor Model Cumulative Residual Re
    ax.plot((1 + pca_residuals[ticker_to_plot]).cumprod() - 1, label='PCA Factor Model Cumulative Residual Return')
    plt.xlabel('Date')
    plt.ylabel('Residual Cumulative Return')
    plt.title(f'In-Sample Factor Model Residual Cumulative Returns for {company_name}')
    plt.legend()
    ax.yaxis.set_major_formatter(mpl_ticker.PercentFormatter(xmax=1))
    plt.show()
```

```
/var/folders/2_/k5xqm0fs6b31lksly6_j661w0000gn/T/ipykernel_29091/3533843098.py:42: FutureWarning: DataFrame.groupby
with axis=1 is deprecated. Do `frame.T.groupby(...)` without axis instead.
  factors = returns.groupby(sector_map, axis=1).mean()
(3443, 207) (3443, 207) (207,)
(3443, 207) (3443, 207) (207,)
Sector composition matrix error (should be less than 1e-10): 1.2e+03
PCA composition matrix error (should be less than 1e-10): 5e-12
Sector Factor Model MAE: 0.010555
PCA Factor Model MAE: 0.010357
(3443, 207)
(3443, 207)
(3443, 207)
(3444, 223)
(3443,) (3443,) (3443,)
```

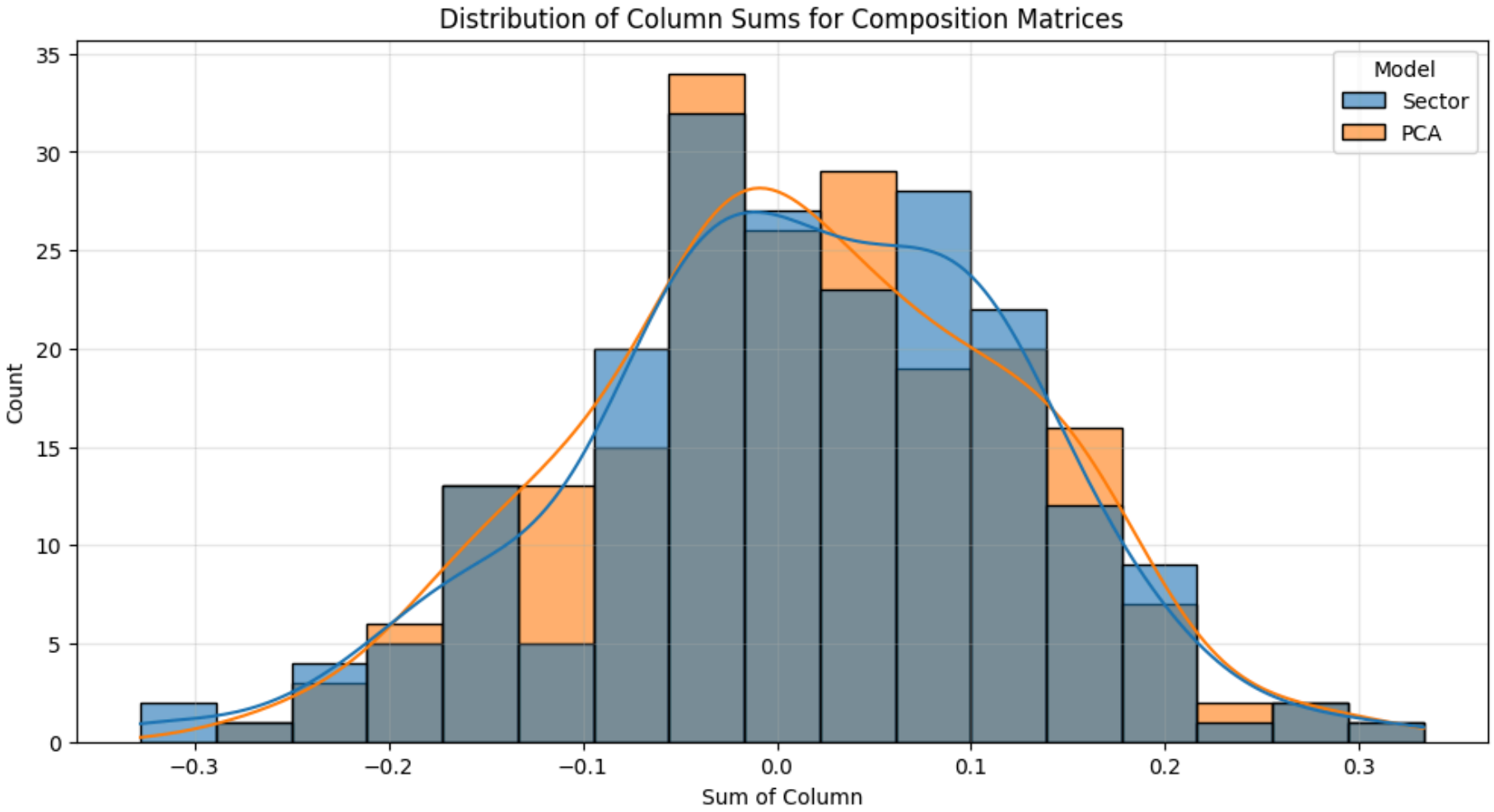

In-Sample Factor Model Predictions vs. True Cumulative Returns for SOFTBANK.GROUP.CORP.
Sector Factor Model Prediction
PCA Factor Model Prediction
True Cumulative Return
Cumulative Return
600%
500%
400%
300%
200%
100%
0%
−100%
2008
2010
2012
2014
2016
2018
2020
Date

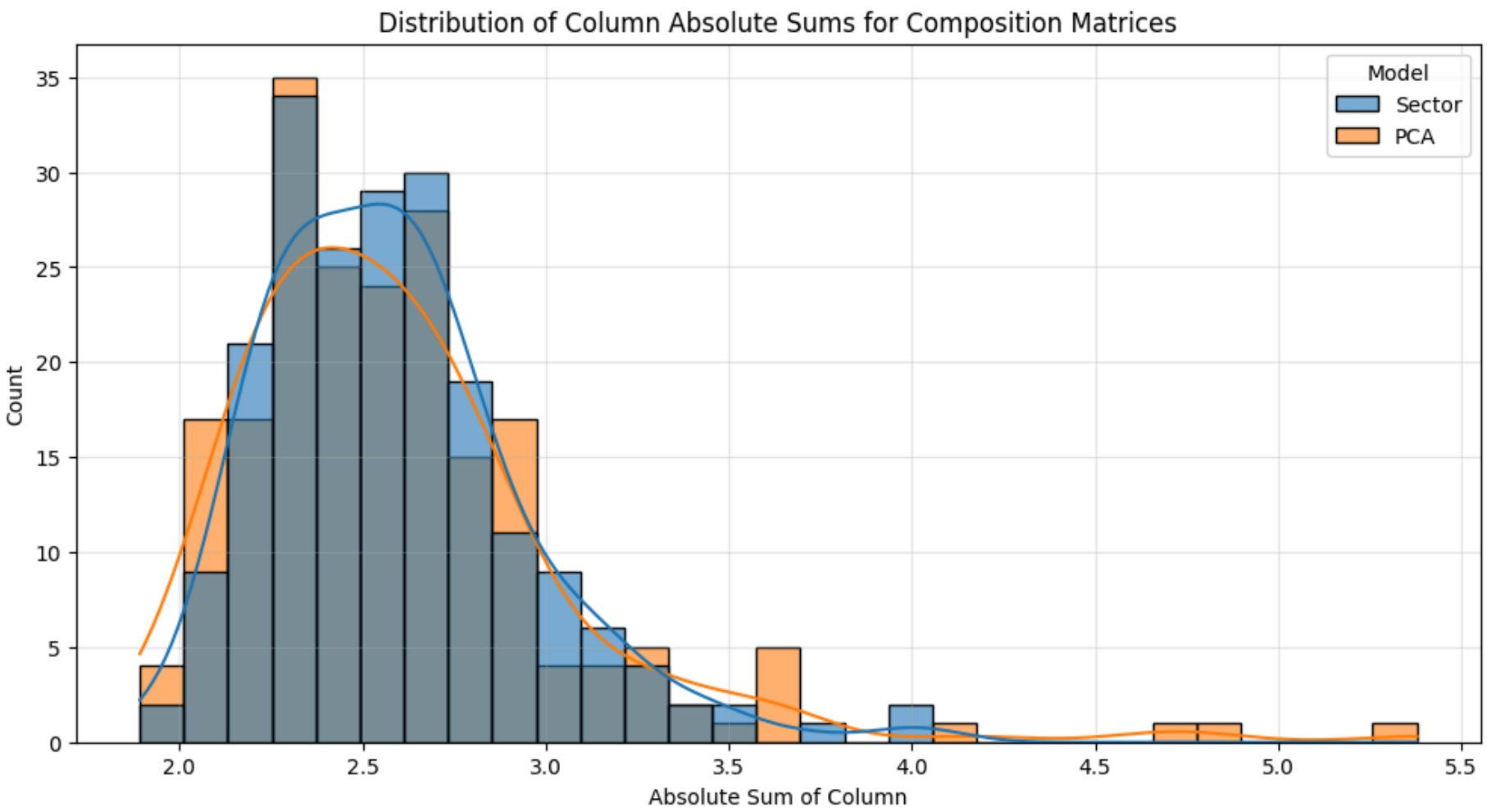


Now we'll plot the column sums of the composition matrices. These correspond to the dollar net exposure of each sector and PCA residual. We'll then plot the absolute column sums of the composition matrices. These correspond to the leverage of each sector and PCA residual.

In [36]:
```
# Calculate column sums for both matrices
sector_sums = (sector_comp_mtx).sum(axis=0)
pca_sums = (pca_comp_mtx).sum(axis=0)

# Create a DataFrame for plotting
hist_data = pd.DataFrame({
```

```
    'Absolute Sum': np.concatenate([sector_sums, pca_sums]),
    'Model': ['Sector']*len(sector_sums) + ['PCA']*len(pca_sums)
})

# Plot histograms
plt.figure(figsize=(12, 6))
sns.histplot(data=hist_data, x='Absolute Sum', hue='Model', kde=True, alpha=0.6)
plt.title('Distribution of Column Sums for Composition Matrices')
plt.xlabel('Sum of Column')
plt.ylabel('Count')
plt.grid(alpha=0.3)
plt.show()

# Calculate column sums for both matrices
sector_sums = np.abs(sector_comp_mtx).sum(axis=0)
pca_sums = np.abs(pca_comp_mtx).sum(axis=0)

# Create a DataFrame for plotting
hist_data = pd.DataFrame({
    'Absolute Sum': np.concatenate([sector_sums, pca_sums]),
    'Model': ['Sector']*len(sector_sums) + ['PCA']*len(pca_sums)
})

# Plot histograms
plt.figure(figsize=(12, 6))
sns.histplot(data=hist_data, x='Absolute Sum', hue='Model', kde=True, alpha=0.6)
plt.title('Distribution of Column Absolute Sums for Composition Matrices')
plt.xlabel('Absolute Sum of Column')
plt.ylabel('Count')
plt.grid(alpha=0.3)
plt.show()
```


Distribution of Column Sums for Composition Matrices
Model
Sector
PCA
Count
Sum of Column
35
30
25
20
15
10
5
0
−0.3
−0.2
−0.1
0.0
0.1
0.2
0.3



Finally, we'll explore a few questions.

Part 1

The sector factor model should in theory mostly residualize assets against other assets in the same sector or industry. Make a rough argument, supported by the data, about why this is or is not systematically true.

One way to do this is the following:

1. Make a new dataframe from the absolute weights of the composition matrix and replace the index names and column names with the sector of each stock.
2. Sort the rows and columns by sector.
3. Set the diagonal entries of this dataframe to zero.
4. Make a heatmap of this dataframe using seaborn, with the sector names on the x and y axis (shown appropriately, i.e. one sector name label for the entire block of sector weights).
5. (Optional) Add dashed lines to mark the boundaries between different sectors. This produces a nice Mondrian-esque heatmap that makes it easier to delineate the sectors.

If the sector factor model does indeed mostly residualize assets against other assets in the same sector or industry, then the heatmap should look somewhat block-diagonal (with a fair bit of noise), where each block corresponds to a different sector.

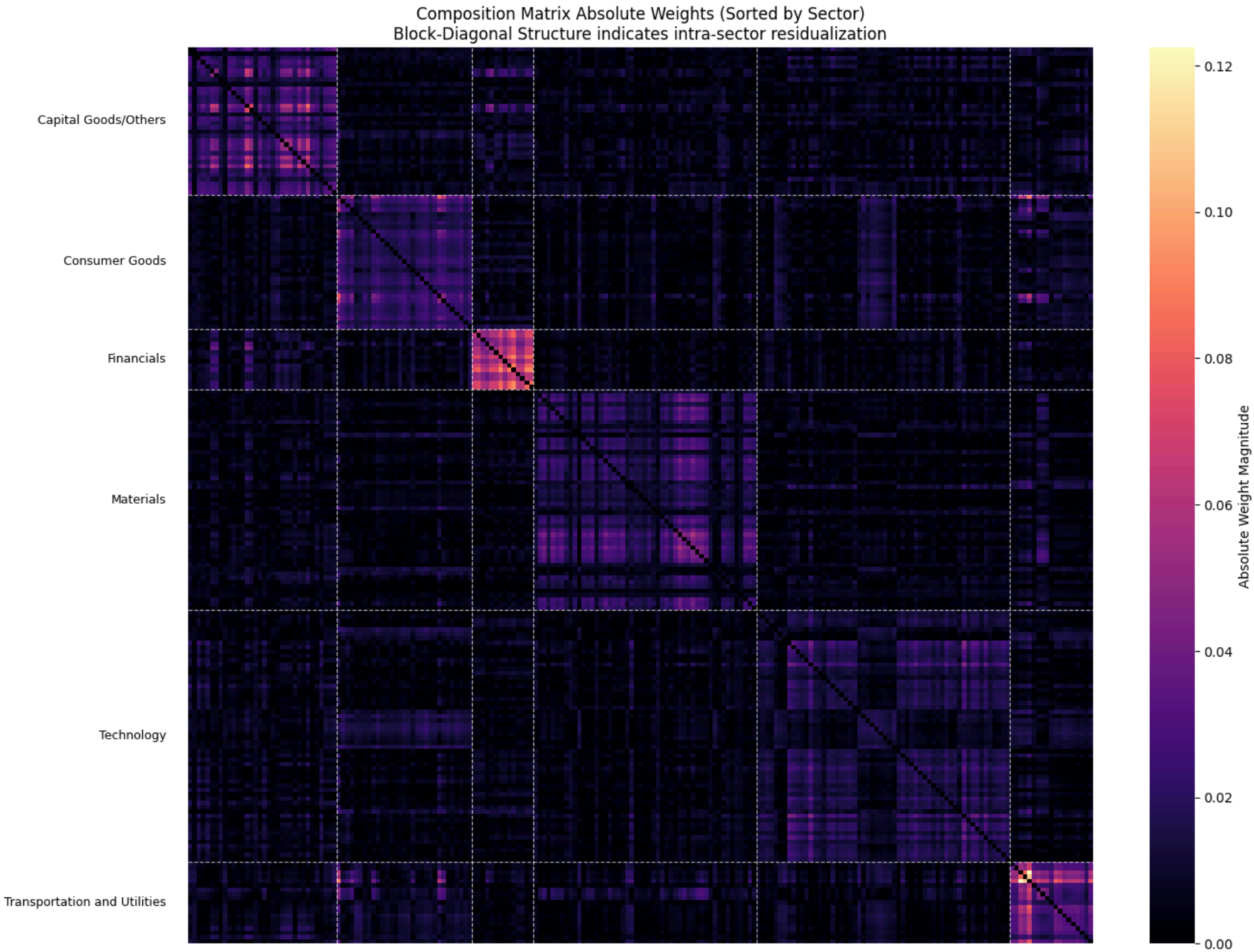
Make sure to use the `valid_returns` throughout, and only take the sectors from the metadata for the stocks in the `valid_stocks`.

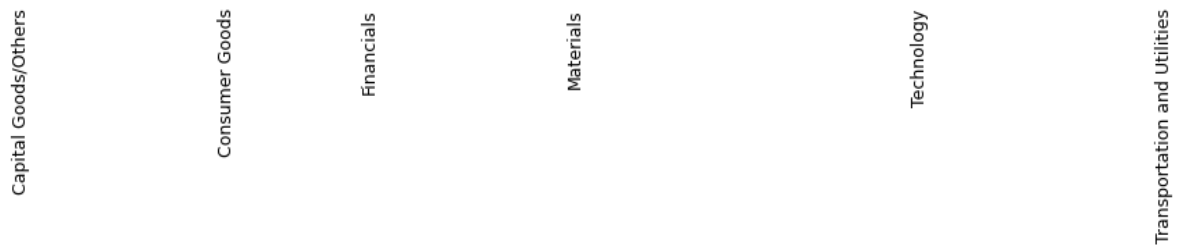
Answer:
The sector factor model does seem to mostly residualize assets against other assets in the same sector or industry. However, there are some exceptions to this, as can be seen in the heatmap below. For example, 'Consumer Goods' and 'Transportation and Utilities' have some cross-sector weights which is maybe to be expected as they are both related to consumer spending. The most significant weights within a sector is the financial sector, the model thinks that financial stocks are very correlated with each other. Some sectors seem to be not very localized like 'Materials' and 'Technology', although they are somewhat localized but exhibit significanly less correlation within the sector compared to other sectors.

In [37]:
```
# 1. Create a dataframe of absolute weights
heatmap_df = pd.DataFrame(
    sector_comp_mtx.values if hasattr(sector_comp_mtx, "values") else sector_comp_mtx,
    index=valid_returns.columns,
    columns=valid_returns.columns
)
heatmap_df = heatmap_df.abs()

# 2. Sort rows and columns by Sector
current_sectors = metadata.loc[heatmap_df.index, 'Sector']
sorted_stocks = current_sectors.sort_values().index
```

```python
# Reorder the dataframe
heatmap_df = heatmap_df.loc[sorted_stocks, sorted_stocks]
sorted_sectors = current_sectors.loc[sorted_stocks]

# 3. Set diagonal to zero (to ignore self-correlation)
np.fill_diagonal(heatmap_df.values, 0)

# 4. & 5. Plot Heatmap with Mondrian-style boundaries
fig, ax = plt.subplots(figsize=(14, 12))

# Plot the heatmap
sns.heatmap(
    heatmap_df,
    cmap='magma',
    xticklabels=False,
    yticklabels=False,
    cbar_kws={'label': 'Absolute Weight Magnitude'},
    ax=ax
)

# Calculate boundaries between sectors
sector_sizes = sorted_sectors.groupby(sorted_sectors, sort=False).size()
boundaries = np.cumsum(sector_sizes)
boundaries = np.insert(boundaries.values, 0, 0)

# Add dashed lines and labels
unique_sectors = sector_sizes.index

for i, sector_name in enumerate(unique_sectors):
    start = boundaries[i]
    end = boundaries[i+1]
    mid = (start + end) / 2

    # White Dashed Lines
    ax.hlines(end, *ax.get_xlim(), colors='white', linestyles='--', linewidth=0.8, alpha=0.7)
    ax.vlines(end, *ax.get_ylim(), colors='white', linestyles='--', linewidth=0.8, alpha=0.7)

    # Labels
    ax.text(mid, len(heatmap_df) + 5, sector_name, ha='center', va='top', rotation=90, fontsize=9, color='black')
    ax.text(-5, mid, sector_name, ha='right', va='center', fontsize=9, color='black')
```

```
plt.title('Composition Matrix Absolute Weights (Sorted by Sector)\nBlock-Diagonal Structure indicates intra-sector
plt.tight_layout()
plt.show()
```


Composition Matrix Absolute Weights (Sorted by Sector)
Block-Diagonal Structure indicates intra-sector residualization
Capital Goods/Others
Consumer Goods
Financials
Materials
Technology
Transportation and Utilities
0.12
0.10
0.08
0.06
0.04
0.02
0.00
Absolute Weight Magnitude



Part 2

Although the factors of the PCA factor model are often uninterpretable, the first few factors usually have some meaning. In fact, the first factor generally especially meaningful weights on the returns: it is usually said to approximate a broad market factor. In this case, we'll define "broad market factor" to be the equally-weighted index of (valid) returns.

In this question, you'll provide some evidence for and extensions of this common observation. Do the following:

1. Examine the first factor of the PCA model and make an argument that it approximates the market factor. This argument can be made by comparisons with the factor time series, by examining the factor weights of the first factor (including how they relate to the metadata), or by some other means. Feel free to normalize the factor weights as you see fit.
2. Examine the second factor and third factor of the PCA model. Can you interpret them? Do they look like any particular time series? Do they load on specific industries or sectors?

Make sure to use the `valid_returns` throughout.

Answer:
From the plots below we see that the weights are all positive for the first principal component. Since all stocks has a positive weight it indicates that it approximates the market factor. The range of values in the first principal component is also smaller than the others about 0.08 compared to 0.2 and 0.4 for the second and third principal components respectively. This indicates that the stocks contribute more equally to the first principal component. Indeed, plotting the equal weighted market against the return of the first eigenportfolio shows that they are highly correlated.

For the third principal component we see that it has positive loads on the sectors transportation and utility and financials and negative loads on technology and materials. The first category of sectors are usually associated with lower pricing multiples. For instance, they tend to have higher book to market and lower pe-ratios. The opposite is true for the negatively loaded sectors. This might indicate that

the second principal component approximates the value vs. growth premium. This is similar to the high minus low factor used in the famous Fama-French three-factor model.

The second principal component is even harder to interpret. After reading a bit online it seems that our seond principal component might approximate yen sensitivity or size (small minus big, another factor used in the Fama-French model). Since we do not have data on the yen or the market cap of the stocks it is hard to make a conclusion about this.

In [42]:
```
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
import pandas as pd

pca = PCA(n_components=1)
pca.fit(valid_returns)
pc1 = pca.transform(valid_returns).ravel()
pc1 = pd.Series(pc1, index = valid_returns.index, name = "PC1")

#Factor Time Seires:
market_ret = valid_returns.mean(axis = 1)
pc1_standardize = (pc1 - pc1.mean()) / (pc1.std())
market_standardize = (market_ret - market_ret.mean()) / (market_ret.std())

diff = pc1_standardize.cumsum() - market_standardize.cumsum()

plt.figure()
plt.plot(diff)
plt.legend()
plt.title("Difference in Cumulative Returns between Market and PC1 loadings")
plt.show()

#Analysis of Factor Weights:
load1 = pd.Series(pca.components_[0], index = valid_returns.columns, name = "PC1_loading")
load1 = load1 / load1.sum()

print((load1 > 0).mean())
print(load1.describe())
load1.hist(bins=50)
plt.title("PC1 Loading Historgram")
plt.show()
```

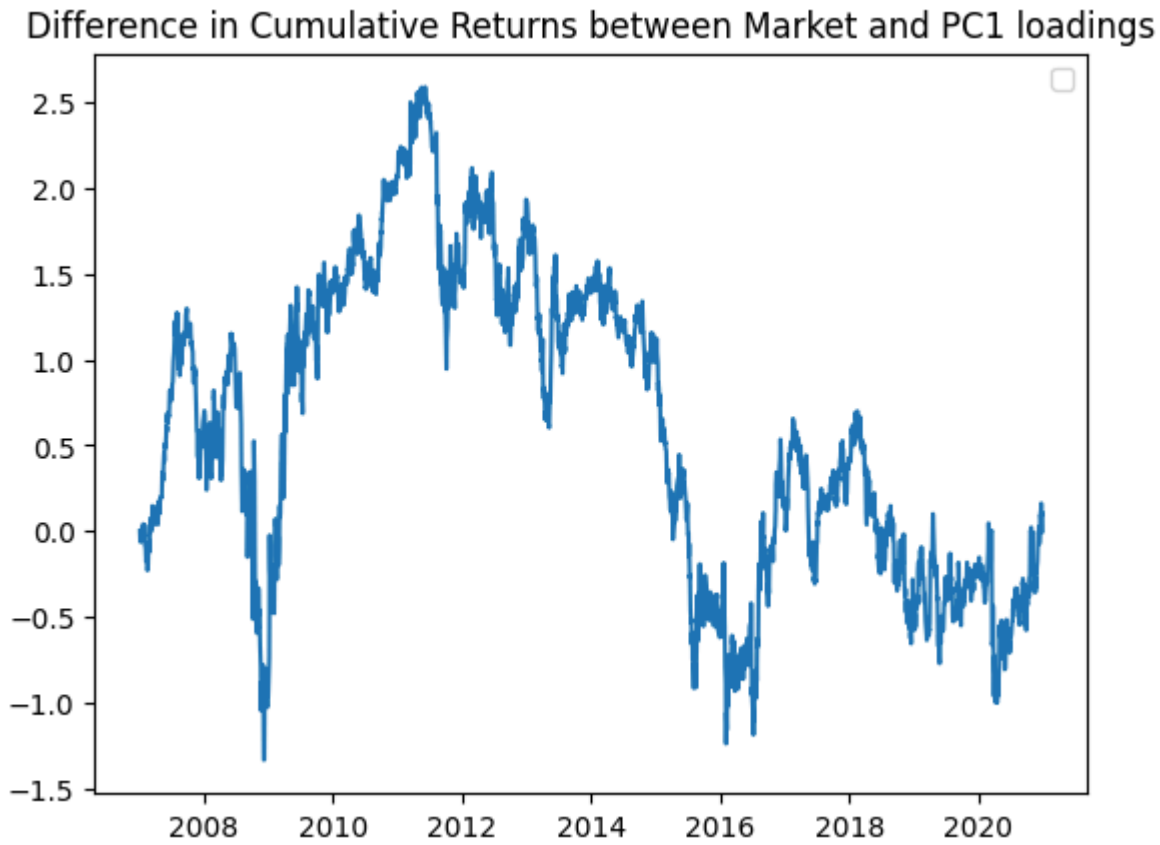
```
#2nd + 3rd Factors:
pca = PCA(n_components=3)
pca.fit(valid_returns)

factors = pca.transform(valid_returns)
pc2 = pd.Series(factors[:, 1], index = valid_returns.index, name = "PC2")
pc3 = pd.Series(factors[:, 2], index = valid_returns.index, name = "PC3")

pc2_standardize = (pc2 - pc2.mean()) / (pc2.std())
pc3_standardize = (pc3 - pc3.mean()) / (pc3.std())

plt.plot(pc2_standardize)
plt.title("Standardized PC2 Time Series")
plt.show()

plt.plot(pc3_standardize)
plt.title("Standardized PC3 Time Series")
plt.show()

load2 = pd.Series(pca.components_[1], index = valid_returns.columns, name = "PC2_Loading")
load3 = pd.Series(pca.components_[2], index = valid_returns.columns, name = "PC3_Loading")

print((load2 > 0).mean())
print((load3 > 0).mean())
print(load2.describe())
print(load3.describe())



# Begin your answer here; Feel free to add code and markdown cells as needed.
valid_prices, valid_returns, valid_stocks = select_asset_universe(prices, returns, prices.index[-1], config)


pca = PCA(3)
factors = pca.fit_transform(valid_returns) # sklearn transforms it into a covariance matrix (we don't have to do it
factors = pd.DataFrame(factors, index=valid_returns.index)
factor_weights = pd.DataFrame(pca.components_.T, index=valid_returns.columns)
```

```python
# ---- Ensure proper column names ----
factor_weights.columns = ['PC1', 'PC2', 'PC3']

# ---- Explained variance (from PCA object) ----
explained_variance = pca.explained_variance_ratio_

# ---- Plot sorted scatter for each principal component ----
fig, axes = plt.subplots(3, 1, figsize=(14, 10), sharex=False)

for i, pc in enumerate(factor_weights.columns):
    # Sort companies by factor loading
    sorted_loadings = factor_weights[pc].sort_values()

    axes[i].scatter(
        range(len(sorted_loadings)),
        sorted_loadings.values,
        s=15
    )

    axes[i].axhline(0, linestyle='--', linewidth=1)

    axes[i].set_title(
        f'{pc} | Explained Variance: {explained_variance[i]:.2%}'
    )

    axes[i].set_xlabel('Companies (sorted)')
    axes[i].set_ylabel('Factor Weight')

plt.tight_layout()
plt.show()



pca_weights = pca_factor_model.factor_weights
pca_weights.columns = [f'Factor_{i}' for i in range(6)]
market_index = valid_returns.mean(axis=1)


# 1. EXAMINE FACTOR 1
raw_weights_f1 = pca_weights['Factor_0'].copy()
```

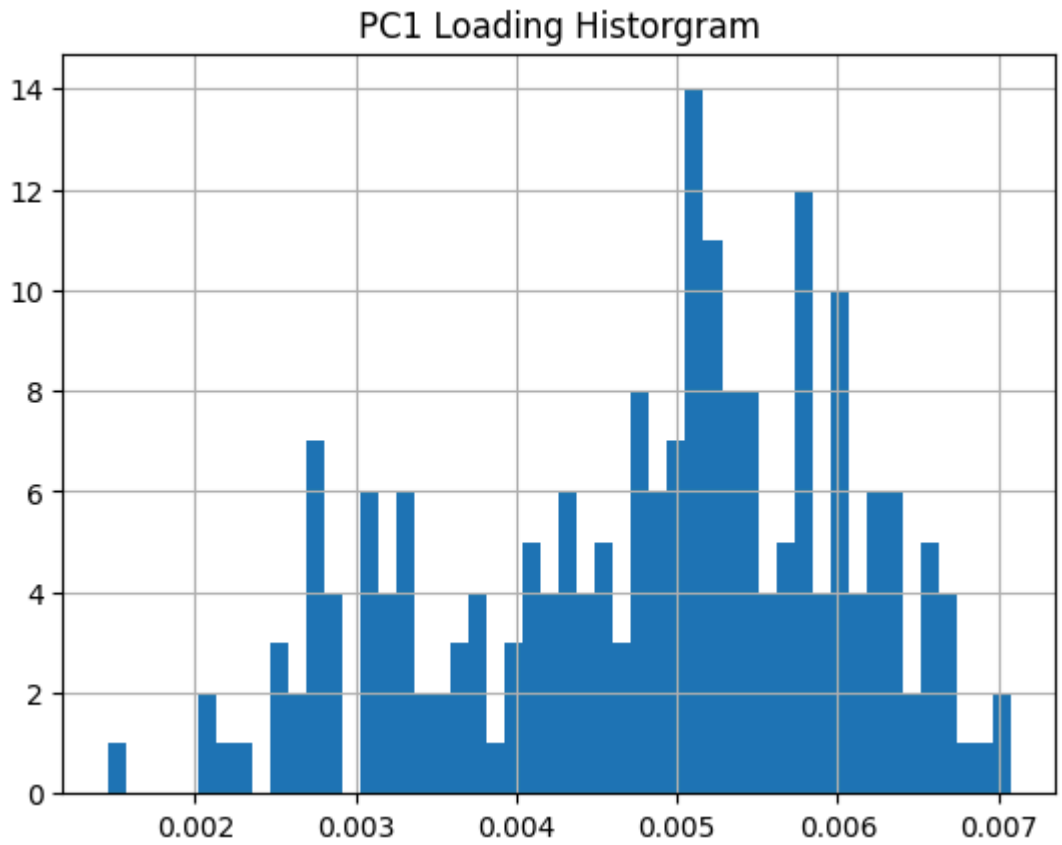
```python
# CHECK POLARITY:
# PCA signs are arbitrary. Check correlation with the market.
# If negative, flip the raw weights first.
temp_factor = valid_returns @ raw_weights_f1
if temp_factor.corr(market_index) < 0:
    raw_weights_f1 = -raw_weights_f1

# NORMALIZE WEIGHTS:
# We divide by the sum of weights so they sum to 1.0 (like a real portfolio).
# This prevents the "14x leverage" crash you saw.
normalized_weights_f1 = raw_weights_f1 / raw_weights_f1.sum()
factor1_ret = valid_returns @ normalized_weights_f1

print(f"--- FACTOR 1 ANALYSIS ---")
print(f"Correlation with Market (Equal-Weighted): {factor1_ret.corr(market_index):.4f}")
print(f"Percentage of positive weights: {(normalized_weights_f1 > 0).mean() * 100:.1f}%")

# Plot 1: Cumulative Returns Comparison
plt.figure(figsize=(14, 5))
plt.subplot(1, 2, 1)
plt.plot((1 + factor1_ret).cumprod() - 1, label='PCA Factor 1 (Normalized)', linewidth=2)
plt.plot((1 + market_index).cumprod() - 1, label='Equal-Weighted Market', linestyle='--', linewidth=2, alpha=0.7)
plt.title('Validation: Factor 1 vs Market (Scaled)')
plt.ylabel('Cumulative Return')
plt.legend()
plt.grid(True, alpha=0.3)

# Plot 2: Weights Histogram
plt.subplot(1, 2, 2)
sns.histplot(normalized_weights_f1, kde=True, color='green')
total_n_stocks = len(normalized_weights_f1)

plt.axvline(1/total_n_stocks, color='red', linestyle='--')
plt.title('Factor 1 Weights Distribution (normalized)')
plt.xlabel('Weight Magnitude')
plt.tight_layout()
plt.show()



# 2. EXAMINE FACTORS 2 & 3
```

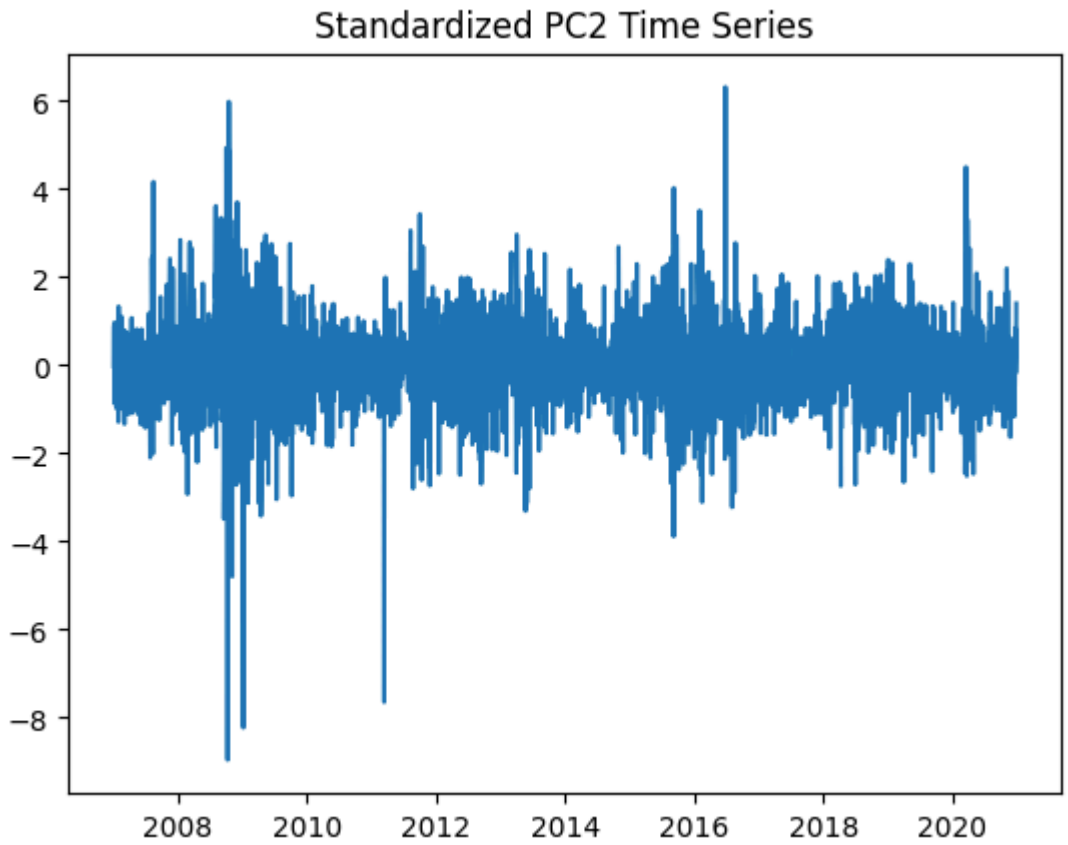
```python
def analyze_factor_by_sector(factor_col_name, plot_title):
    weights = pca_weights[factor_col_name].copy()

    df = pd.DataFrame({'Weight': weights})
    df['Sector'] = metadata.loc[df.index, 'Sector']
    sector_order = df.groupby('Sector')['Weight'].median().sort_values(ascending=False).index

    plt.figure(figsize=(14, 6))
    sns.boxplot(x='Sector', y='Weight', data=df, order=sector_order, palette='coolwarm')
    plt.axhline(0, color='black', linestyle='-', linewidth=1)
    plt.xticks(rotation=90)
    plt.title(f'{plot_title} Loadings by Sector')
    plt.grid(True, axis='y', alpha=0.3)
    plt.tight_layout()
    plt.show()

    print(f"\n--- {plot_title} INTERPRETATION ---")
    print(f"Most Positive Loading: {sector_order[0]}")
    print(f"Most Negative Loading: {sector_order[-1]}")

# Run analysis for Factor 2 and Factor 3
analyze_factor_by_sector('Factor_1', "PCA Factor 2")
analyze_factor_by_sector('Factor_2', "PCA Factor 3")
```

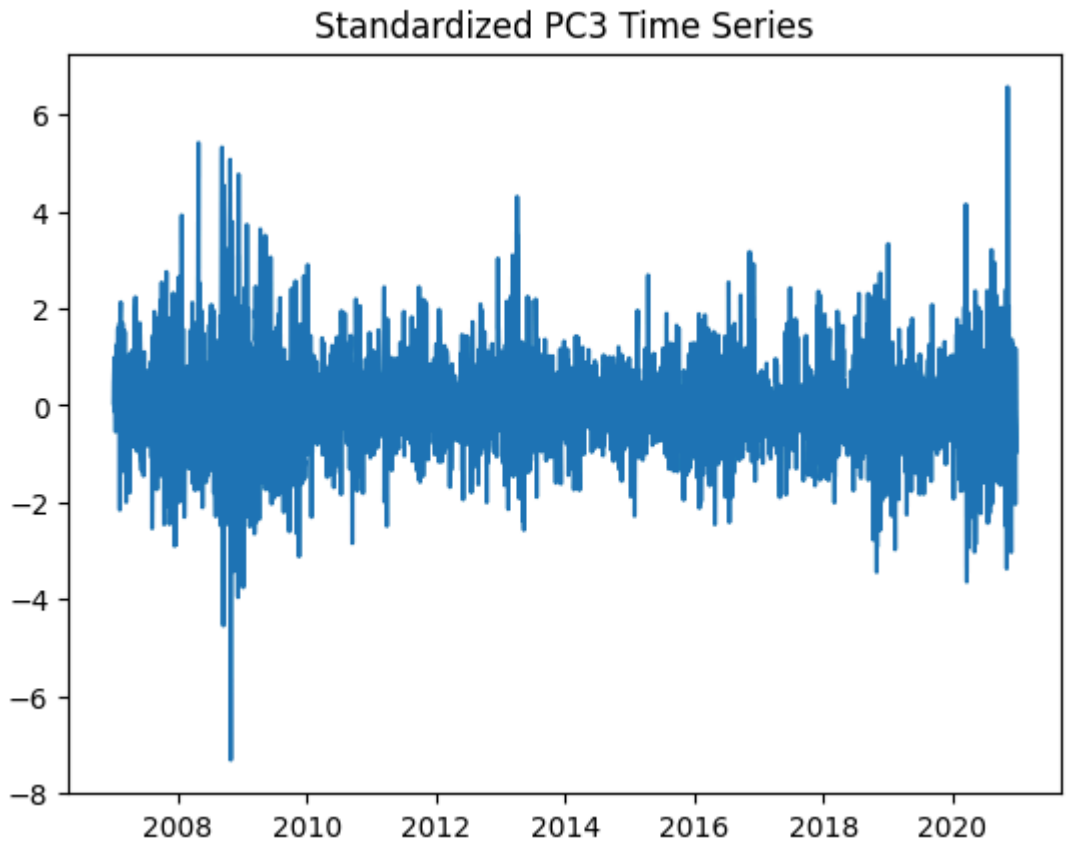
```
/var/folders/3t/5vvh8l5x48s7tyvbx4b6v9xr0000gn/T/ipykernel_48820/3046148465.py:20: UserWarning: No artists with labe
ls found to put in legend.  Note that artists whose label start with an underscore are ignored when legend() is call
ed with no argument.
  plt.legend()
```



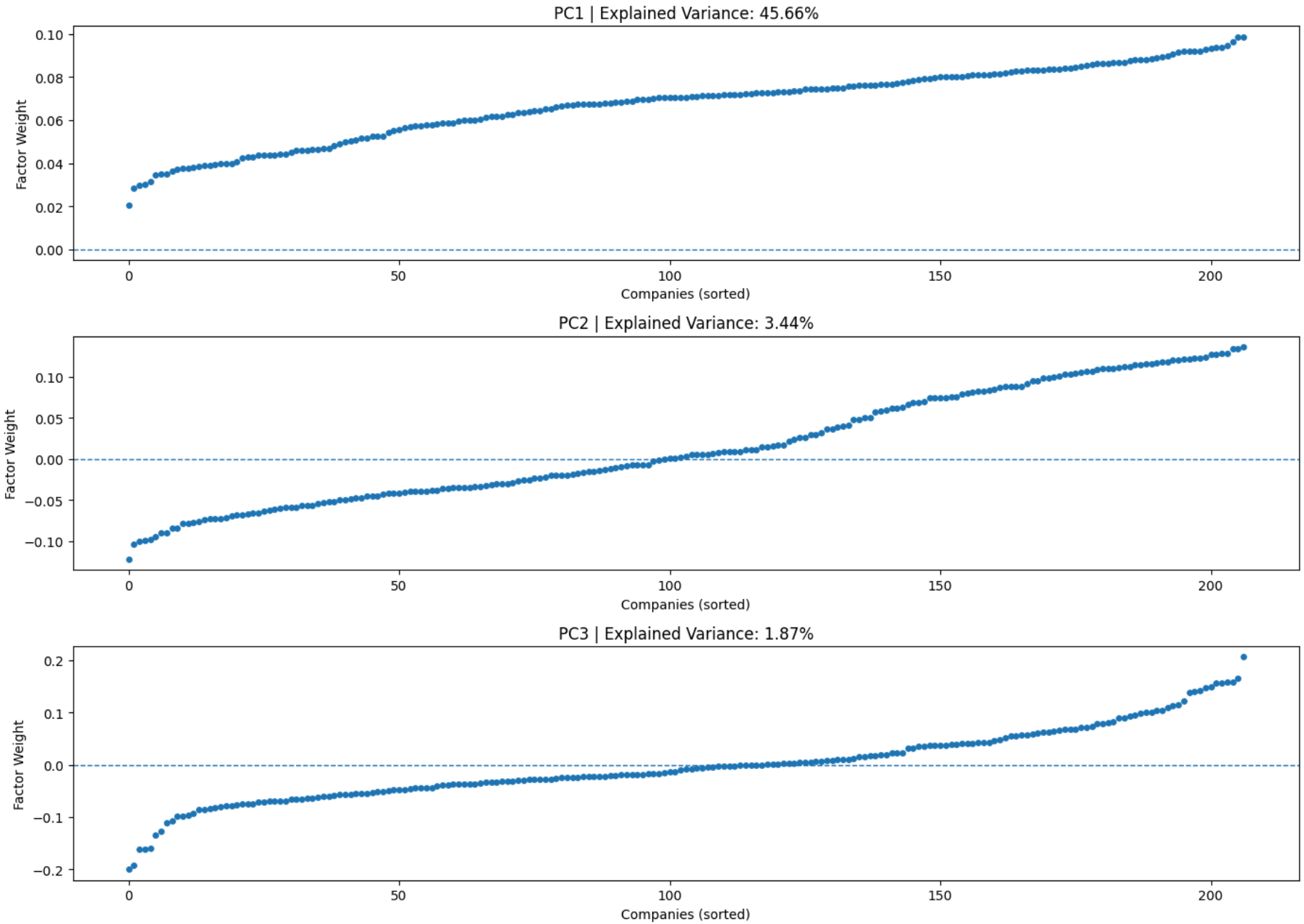

```
1.0
count    207.0000
mean       0.0048
std        0.0012
min        0.0015
25%        0.0041
50%        0.0051
75%        0.0058
max        0.0071
Name: PC1_loading, dtype: float64
```


PC1 Loading Historgram
14
12
10
8
6
4
2
0
0.002
0.003
0.004
0.005
0.006
0.007

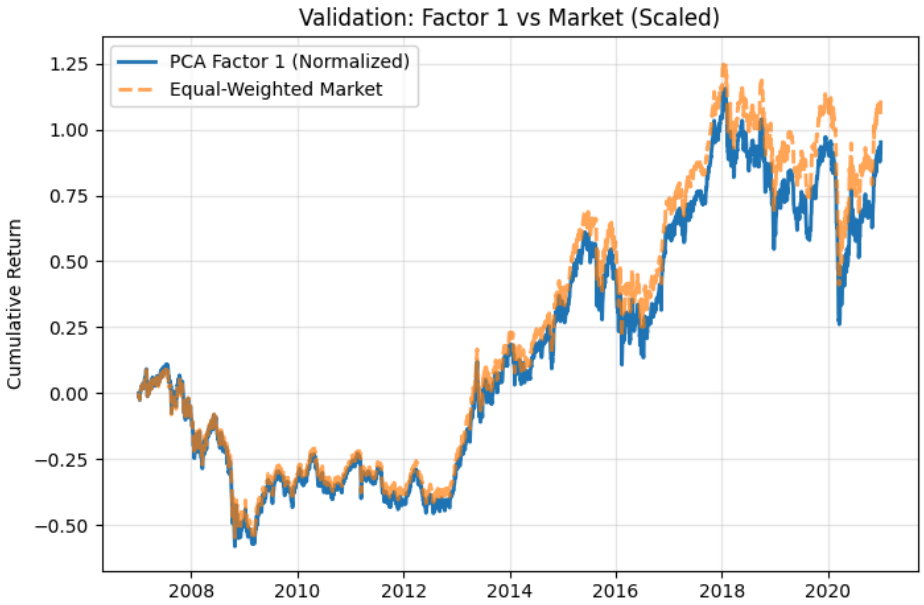

Standardized PC2 Time Series
6
4
2
0
−2
−4
−6
−8
2008
2010
2012
2014
2016
2018
2020

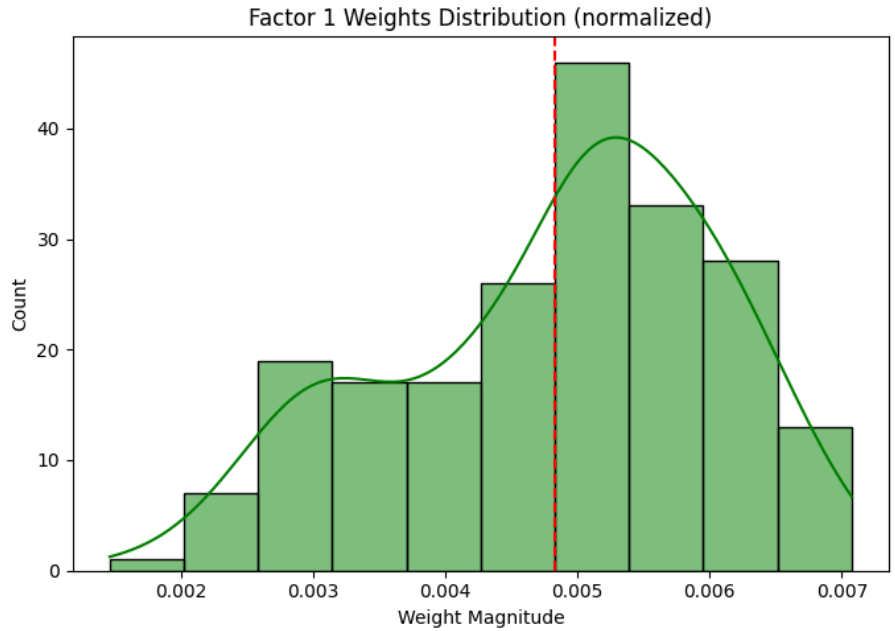

Standardized PC3 Time Series
6
4
2
0
−2
−4
−6
−8
2008
2010
2012
2014
2016
2018
2020

```
0.5169082125603864
0.43478260869565216
count    207.0000
mean       0.0145
std        0.0681
min       -0.1224
25%       -0.0404
50%        0.0030
75%        0.0797
max        0.1367
Name: PC2_Loading, dtype: float64
count    207.0000
mean      -0.0017
std        0.0697
min       -0.2001
25%       -0.0463
50%       -0.0075
75%        0.0411
max        0.2062
Name: PC3_Loading, dtype: float64
```

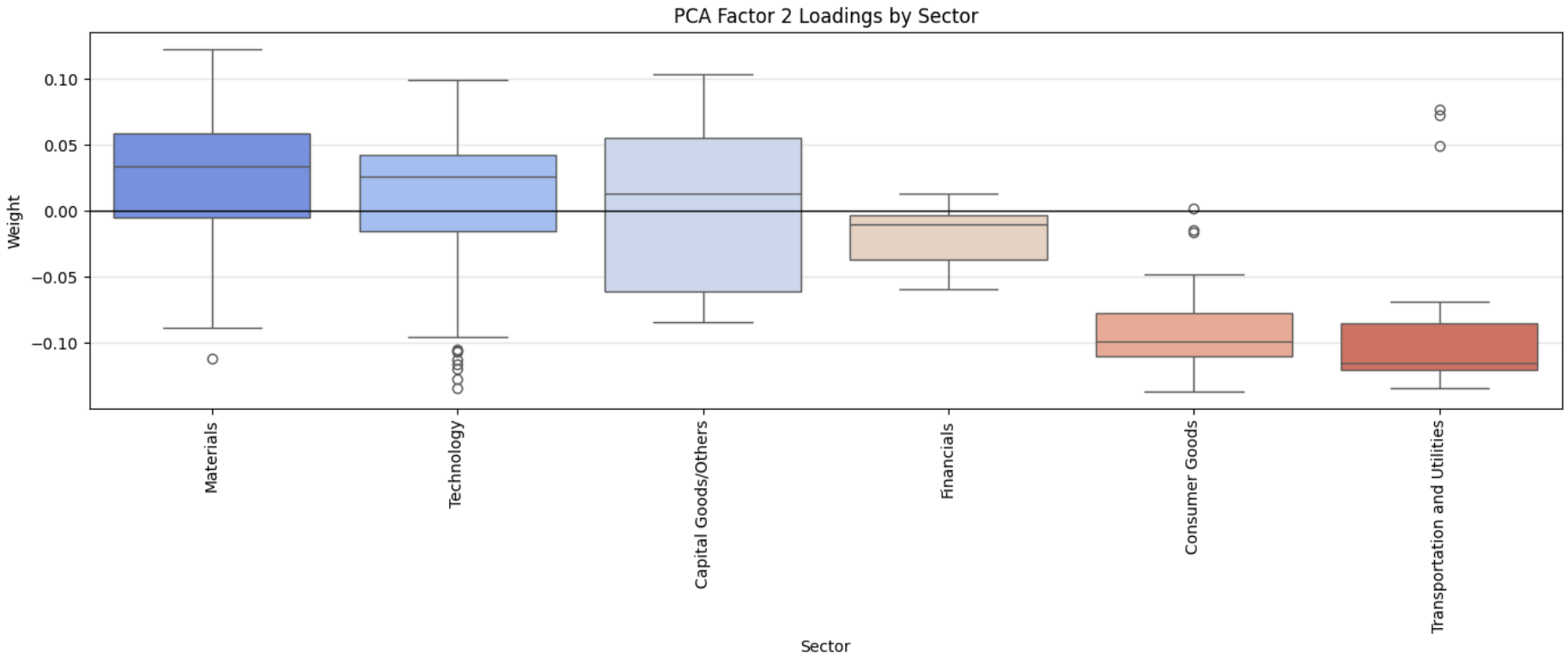

PC1 | Explained Variance: 45.66%
Factor Weight
Companies (sorted)
PC2 | Explained Variance: 3.44%
Factor Weight
Companies (sorted)
PC3 | Explained Variance: 1.87%
Factor Weight
Companies (sorted)

```
--- FACTOR 1 ANALYSIS ---
Correlation with Market (Equal-Weighted): 0.9981
Percentage of positive weights: 100.0%
```



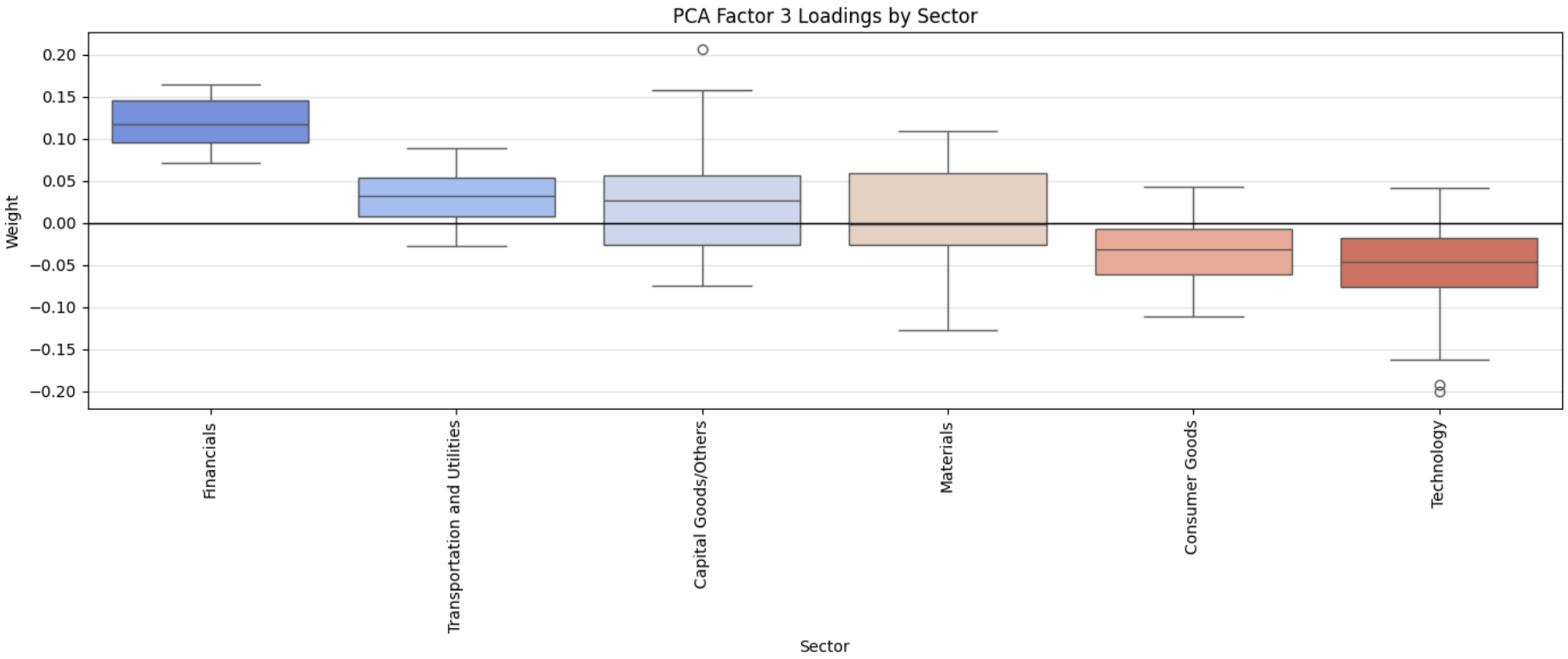



```
/var/folders/3t/5vvh8l5x48s7tyvbx4b6v9xr0000gn/T/ipykernel_48820/3046148465.py:166: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `
hue` and set `legend=False` for the same effect.

  sns.boxplot(x='Sector', y='Weight', data=df, order=sector_order, palette='coolwarm')
```




```
--- PCA Factor 2 INTERPRETATION ---
Most Positive Loading: Materials
Most Negative Loading: Transportation and Utilities
```

```
/var/folders/3t/5vvh8l5x48s7tyvbx4b6v9xr0000gn/T/ipykernel_48820/3046148465.py:166: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `
hue` and set `legend=False` for the same effect.

  sns.boxplot(x='Sector', y='Weight', data=df, order=sector_order, palette='coolwarm')
```




```
--- PCA Factor 3 INTERPRETATION ---
Most Positive Loading: Financials
Most Negative Loading: Technology
```

Question 6 (extra credit)

(0.25 bonus points)

One interesting question to explore is to try to identify which factor model produces residuals with stronger mean-reversion. In this question, we'll explore this.

To help you out, we've implemented a class which performs an ADF test on each residual passed in.

In [39]:
```
class NikkeiADFTest():
    def __init__(self, prices: pd.DataFrame):
        self.prices = prices

    def test(self) -> pd.DataFrame:
        results = []
        if isinstance(self.prices, pd.Series):
            adf = arch.unitroot.ADF(self.prices)
            results.append({'ADF Statistic': adf.stat, 'P-Value': adf.pvalue})
        elif isinstance(self.prices, pd.DataFrame):
            for col in self.prices.columns:
                adf = arch.unitroot.ADF(self.prices[col])
                results.append({'Stock': col, 'ADF Statistic': adf.stat, 'P-Value': adf.pvalue})
        else:
            raise ValueError('Input must be a pandas Series or DataFrame')
        return pd.DataFrame(results)
```

Test the normalized residual prices (i.e. the cumulative residual return) of each of the residuals from the sector factor model and from the PCA factor model. Concatenate and compare the results. Which factor model produces more significantly mean-reverting residuals at a significant level of 0.05? You may plot an overlaid histogram of each of the sets of p-values if you like. (Note that we do not correct for multiple testing here, but that could potentially be a good idea in a real analysis.)

One interesting thing to explore here might be to see if the intercept term in the factor model makes a difference in the results. If you did the extra credit, you can test this as well below (though this is not required for the extra credit).

Make sure to use the `valid_returns` throughout.

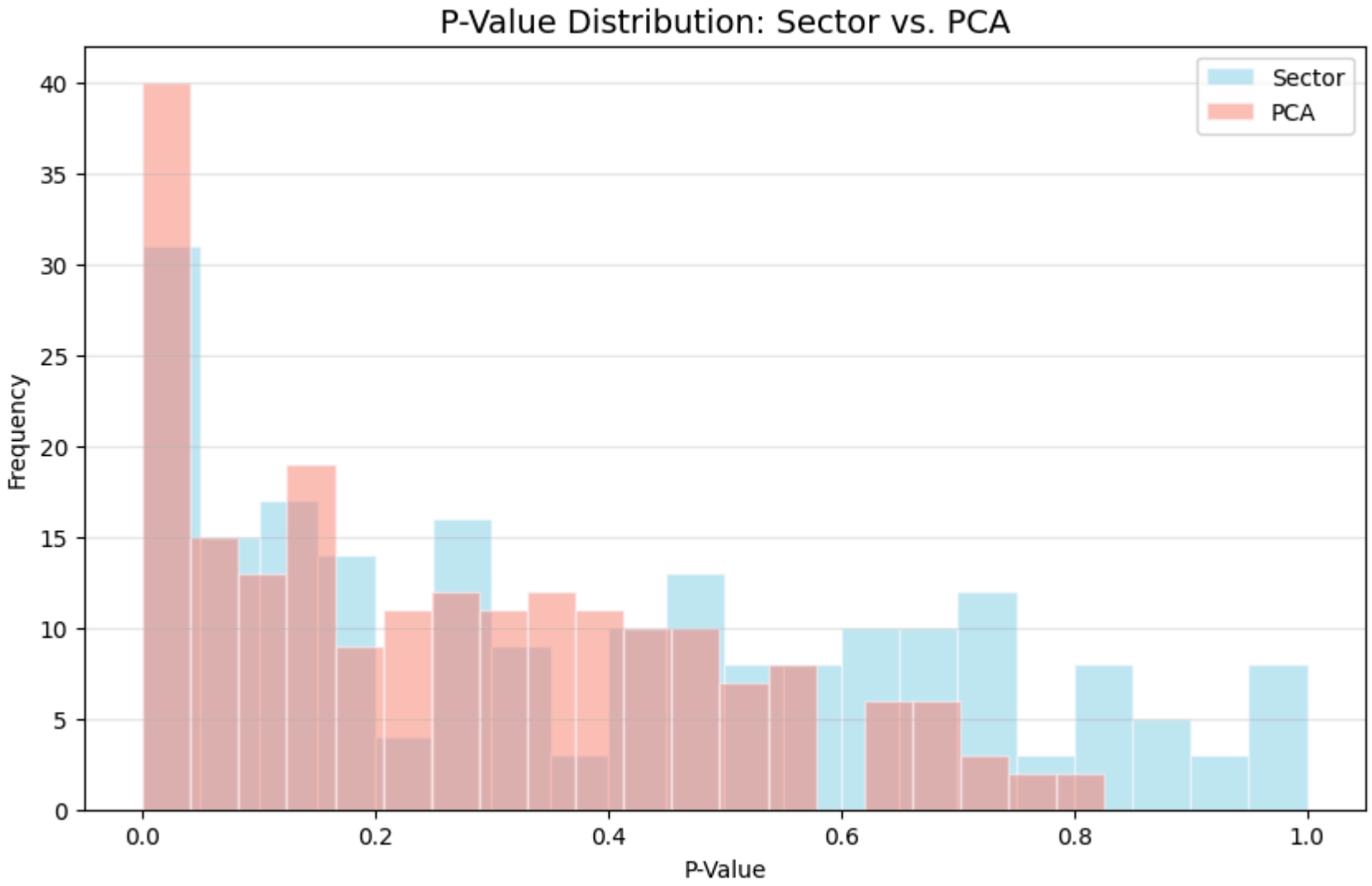
Answer:
As we can see from the below analysis it seems that pca produces more significant mean-reverting residuals. 20.8% of the series produced by pca have significant mean reversion compared to only 15% for sector. From the histogram it is also clear that pca tend to produce more significant mean-reverting residuals than sector.

In [43]:
```
# Begin your answer here; Feel free to add code and markdown cells as needed.
# Begin your answer here; Feel free to add code and markdown cells as needed.
use_intercept = False
```

```python
# Select the universe of assets to be used in the factor models
config = {
    'LOOKBACK_PERIOD': len(returns) - 1,
    'FILTER_MAX_ABS_RETURN': 0.5
}
valid_prices, valid_returns, valid_stocks = select_asset_universe(prices, returns, prices.index[-1], config)

# Fit the industry factor model and get residuals
sector_factor_model = NikkeiSectorFactorModel(metadata, intercept=use_intercept)
sector_residuals, sector_predictions, sector_comp_mtx = sector_factor_model.fit(valid_returns)

# Fit the PCA factor model and get residuals
pca_factor_model = NikkeiPCAFactorModel(num_factors=6, intercept=use_intercept)
pca_residuals, pca_predictions, pca_comp_mtx = pca_factor_model.fit(valid_returns)

tot_result = pd.DataFrame()
for company_name in valid_returns.columns:
    cumulative_return_sector = (1 + sector_residuals[company_name]).cumprod() - 1
    cumulative_return_pca = (1 + pca_residuals[company_name]).cumprod() - 1

    sector_test = NikkeiADFTest(cumulative_return_sector)
    pca_test = NikkeiADFTest(cumulative_return_pca)

    sector_result = sector_test.test()
    pca_result = pca_test.test()

    result = pd.concat([sector_result, pca_result])
    result.index = ['Sector', 'PCA']
    tot_result = pd.concat([tot_result, result])


p_sector = tot_result.loc['Sector', 'P-Value']
p_pca = tot_result.loc['PCA', 'P-Value']

count_sector = (p_sector < 0.05).mean()
count_pca = (p_pca < 0.05).mean()

print(f"Percent significant Sector series (p < 0.05): {count_sector*100}%")
print(f"Percent significant PCA series (p < 0.05): {count_pca*100}%")

plt.figure(figsize=(10, 6))
```

```
plt.hist(p_sector, bins=20, alpha=0.5, label='Sector', color='skyblue', edgecolor='white')
plt.hist(p_pca, bins=20, alpha=0.5, label='PCA', color='salmon', edgecolor='white')

# Add styling and labels
plt.title('P-Value Distribution: Sector vs. PCA', fontsize=14)
plt.xlabel('P-Value')
plt.ylabel('Frequency')
plt.legend(loc='upper right')
plt.grid(axis='y', alpha=0.3)

plt.show()
```

```
/var/folders/3t/5vvh8l5x48s7tyvbx4b6v9xr0000gn/T/ipykernel_48820/3533843098.py:42: FutureWarning: DataFrame.groupby
with axis=1 is deprecated. Do `frame.T.groupby(...)` without axis instead.
  factors = returns.groupby(sector_map, axis=1).mean()
Percent significant Sector series (p < 0.05): 14.975845410628018%
Percent significant PCA series (p < 0.05): 20.77294685990338%
```


P-Value Distribution: Sector vs. PCA
Sector
PCA
Frequency
P-Value
0
5
10
15
20
25
30
35
40
0.0
0.2
0.4
0.6
0.8
1.0